scientific reports

Check for updates

OPEN

Fast moving table tennis ball tracking algorithm based on graph neural network

Tianjian Zou[1], Jiangning Wei[1,3], Bo Yu[1,3], Xinzhu Qiu[1,3], Hao Zhang[2,3], Xu Du[2,3] & Jun Liu[1,3]✉

The key object tracking in sports video scenarios poses a pivotal challenge in the analysis of sports techniques and tactics. In table tennis, due to the small size and rapid motion of the ball, identifying and tracking the table tennis ball through video is a particularly arduous task, where the majority of existing detection and tracking algorithms struggle to meet the practical application requirements in real-world scenarios. To address this issue, this paper proposes a combined technical approach integrating detection and discrimination, tailored to the unique motion characteristics of table tennis. For the detector, we utilize and refine a common video differential detector. As for the discriminator, we introduce GMP (a Graph Max-message Pass Neural Network), which is designed specifically for tracking table tennis balls or similar objects. Furthermore, we enhance an existing dataset for table tennis tracking problems by enriching its scenarios. The results demonstrate that our proposed technical solution performs impressively on both the dataset and the intended real-world environments, showcasing the good scalability of our algorithms and models as well as their potential for application in other scenarios.

Keywords Table tennis, Fast moving object, Object tracking, Object detection, Graph neural network, Sports analytics

The application of AI technology in sports has emerged as one of the recent research hotspots, with AI-based sports analysis already finding its way into various sports such as football, tennis and billiards. These intelligent analysis systems deployed in sports scenarios typically leverage multiple advancements in AI fields as their core technological components, adapting them to cater to the specific needs of sports analysis. For instance, human body key point extraction and pose recognition techniques are employed to identify athletes' movements, while object detection and tracking technologies are utilized to record and analyze match scenarios. Additionally, anomaly detection and recognition techniques based on video and motion image features are used to extract key frames. These technologies aid athletes and coaches in making rational plans and decisions, reducing the cost of training and teaching, and enhancing the popularity of sports, thereby developing and elevating the overall performance level of the sport.

Similarly, table tennis is a widely popular sport globally, and video-based intelligent analysis systems for table tennis can assist a vast number of enthusiasts and athletes in evaluating their tactical characteristics, providing standardized movement guidance[1], and planning scientific training, thereby effectively enhancing their skill levels. Consequently, video-based intelligent analysis systems for table tennis techniques possess broad application prospects and ample demand. Within these intelligent analysis systems, ball tracking is an indispensable key technology. It can assist the system in determining the point of contact[2], delineating the start and end moments of movements to aid in action classification and standardization[3], and supporting the action recognition system in segmenting rounds and scoring. However, the small size, rapid motion, and insignificant and unstable image features of the table tennis ball bring many difficulties to the tracking algorithm. Further more, well-researched target detection and tracking algorithm recently,including pedestrian tracking (primarily focused on larger targets with clear image features), and unmanned aerial vehicle (UAV) detection and tracking algorithms (where targets are small but feature-stable, making motion easier to track) are inapplicable in our scenario for these difficulties. Examples of table tennis balls to be detected and tracked are shown in Figure 1.

In the context of small object detection, research represented by[4]suggests that leveraging contextual information and adjacent video frames can enhance tracking capabilities. On the other hand, studies focusing on

[1]School of Artificial Intelligence, Beijing University of Post and Telecommunications, XiTuCheng Road, 10, Haidian 100082, Beijing, China. [2]Department of Physical Education, Beijing University of Post and Telecommunications, XiTuCheng Road, 10, Haidian 100082, Beijing, China. [3]These authors contributed equally: Jiangning Wei, Bo Yu, Xinzhu Qiu, Hao Zhang, Xu Du and Jun Liu. ✉email: liujun@bupt.edu.cn



Fig. 1. Examples of table tennis balls to be detected and tracked.

high-speed moving object tracking, exemplified by[5], propose the utilization of frame differences and analysis of differential images to separate the target from the background. Inspired by these two perspectives, our proposed approach combines detection and discrimination to track table tennis balls. Similar ideas have been explored in existing research, where[6]and[7]achieved good detection results using similar concepts. However, due to the processing of unstructured data, their discriminators tend to have numerous parameters, complex structures, and slow runtimes. Although existing Graph Neural Networks (GNNs) can handle unstructured data, they do not adapt well to the demands of tracking problems. The vast majority of research employing graph neural networks for tracking tasks has focused on Multiple Object Tracking (MOT) datasets, which emphasize the construction of relationships between targets while rarely considering the presence of erroneous, false, or even uncertain targets. While numerous similar studies have also prioritized handling severe occlusion and target loss scenarios[8], these methods still rely on inferences based on relatively good detection results, rather than operating from the outset under the challenging conditions encountered in table tennis videos. Therefore, we introduce a novel Graph Max-message Pass Neural Network (GMP) as a new discriminator specifically designed for tracking and filtering table tennis balls or similar targets. The overall approach of the proposed algorithm will be detailed in Section 3, with a focus on the analysis of the discriminator GMP in Section 3.5.

To train effective models and validate the efficiency of our method, we build a dataset with rich scenarios based on existing research[5679]and conduct comparisons with various general-purpose algorithms on this dataset. The general-purpose algorithms we compared against include YOLOv8, which performs object detection based directly on single-frame image features[101112], Single Object Tracking (SOT), which relies on user annotations and image features for tracking[13], and a scheme that utilizes the generic Graph Attention Network (GAT)[14] as a discriminator. Our findings indicate that our method outperforms all the comparison algorithms and holds the potential to address other object tracking problems. In Section 4, we will delve into the detailed analysis of the performance of our proposed method. In Section 2, we will review related works, and in Section 5, we will provide a brief summary and outline future research directions.

In practical training and match analysis, our algorithm can directly provide information such as the image location and trajectory of the table tennis ball. These direct data assist the analysis system in segmenting rounds and actions. With subsequent analysis and processing, we can obtain the land point and hit point of the table tennis ball and estimate its approximate speed (shown in Figure 2). During training and matches, by analyzing these data, we can quantify players' stability, offensive, and defensive capabilities, and use the data to arrange corresponding tactics and targeted training. For instance, in training, by analyzing players' ball-striking speed and trajectory direction, coaches can correct their force generation method and angle during ball striking, thereby improving their hitting quality. During matches, coaches can formulate reasonable counter-tactics for their own players based on data such as the trajectory and land point of the opponent's table tennis ball. Compared to commonly used match result analysis, scoring and losing statistics, the results provided by the table tennis tracking algorithm are more precise, detailed, and rigorous.

Related works

In our previous research[6], we leveraged a tree structure to establish contextual relationships among potential table tennis ball targets. Initially, we employed a differential detection method to filter out numerous potential targets, ensuring a sufficiently high recall rate. Subsequently, based on the proposed tree structure and the



Fig. 2. Further analysis of the tracking results of table tennis can yield many useful technical and tactical information (including land point, hit point, approximate ball speed, etc.). In particular, we roughly calculate the average pixel distance of the table tennis ball between a trajectory frame, and then roughly estimate the ball speed of the table tennis ball based on the pixel length of the center line of the table, the actual length (2.74m), and the video frame rate (in this case, the frame rate of the video is 30 fps).

kinematic characteristics of table tennis, we refined the targets to identify the table tennis ball through trajectory analysis associated with the leaf nodes in the tree. This approach achieved certain success in terms of accuracy and adaptability across different scenarios. However, it relied heavily on manually tuned empirical parameters and operated relatively slowly. Consequently, we have further delved into this field and present the method proposed in this thesis. Our related work revolves around four aspects: the significance of table tennis tracking algorithms for intelligent analysis systems, object detection methods, traditional object tracking methods, and graph neural network-based object tracking methods.

The significance of table tennis tracking algorithms for intelligent analysis systems

Currently, there have been a number of studies, achievements, and datasets related to intelligent analysis systems for table tennis[9], yet the tracking of table tennis balls in complex scenarios remains limited. This is due to the high technicality of table tennis, which has led most research to focus on motion guidance and standardization. These studies typically utilize deep learning methods to extract human body keypoints, employing them to model and estimate human poses[15]. Algorithms can then determine the start and end of actions based on the pose skeletons of individuals in video frames[16], classify technical actions[17], and design rules for evaluating technique execution. Similar technical solutions do not require table tennis ball detection as they rely solely on human skeletons and keypoints; however, analyzing actions based only on skeletons remains a challenging task[1819]. This approach is only effective in relatively standardized training environments. In less standardized training and competition scenarios, the movement of the table tennis ball is highly variable[20], and players need to adjust their actions accordingly. Without effective table tennis ball tracking technology, intelligent analysis systems may fail to adapt to such changes, leading to useless or even erroneous analysis and guidance, significantly reducing the accuracy and recall rates of motion guidance[3]. Tracking results for table tennis balls can assist in precisely dividing action stages for motion analysis and guidance, aiding in the assessment of the standardization and rationality of actions.

Object detection methods

With the development of object detection techniques in computer vision, detecting objects directly from images based on their edges, morphology, color, and other features, and regressing the bounding boxes of these objects, has become one of the primary methods for object detection. Representative methods include YOLO and FASTER-RCNN. Among them, YOLO[12]has gained widespread popularity due to its simple structure, strong adaptability, and fast operation[1011]. Previous studies have found that applying these methods directly to table tennis ball detection is not ideal due to the small size and excessive blurring of the ball[7]. Some scholars have used an improved FASTER-RCNN algorithm to detect table tennis balls based on their small size characteristics,

achieving good results[21]. However, there is still room for improvement in terms of the method's performance metrics and adaptability to complex scenarios.

Some scholars have chosen to focus on the rapid movement characteristics and shape of table tennis balls, attempting to extract the rapidly moving balls in the foreground based on differential images[5]. Other studies have tried to directly segment blurred table tennis balls in the foreground using deep learning techniques[229], while some have considered relying on background information from the table tennis table to separate the balls in the foreground[23]. These methods perform well in scenarios with uncomplicated backgrounds, but table tennis sports videos in real-world are full of various noises, background clutter, and interference caused by athlete movements. Under such circumstances, the aforementioned approaches often fail to function properly. Some studies have proposed using local images for detection and excluding most image regions based on the ball's motion characteristics to avoid background interference[24]. However, table tennis sports videos in real-world contain a large number of invalid segments where the ball is not present, which limits the direct application of the aforementioned methods.

Traditional object tracking methods

Since relying solely on single-frame images cannot effectively solve the problem of table tennis ball tracking, a natural approach is to leverage the contextual and spatio-temporal information from videos for detection. The conventional idea behind such methods is to extract all potential candidate regions and targets from images or video frames, aggregate these targets, and then screen them based on image features, motion characteristics, and spatio-temporal information to obtain the final tracking results[25].

In[4], researchers extracted all potential targets from single-frame images and then filtered out the best results based on video context, improving the detection effectiveness for blurred small objects. In[26], researchers successfully tracked vehicles in videos with significant distortion by combining the motion characteristics of vehicles. In[27], researchers achieved billiards tracking and 3D reconstruction by proposing all possible positions and shapes of the cue ball.

Furthermore, table tennis ball tracking can also be considered as part of the Single Object Tracking (SOT) problem[28]. However, due to SOT's heavy reliance on stable image features and feature extraction networks[29], its application to table tennis ball tracking is limited. Nevertheless, in this context, researchers often focus on the Region Proposal Network (RPN) responsible for candidate region generation[13]. The role of this network is to analyze all potential candidate regions and select the optimal tracking results. This aspect of the work essentially involves analyzing and discriminating all potential targets from a global perspective, assessing the confidence of each target based on prior information, and filtering out the results.

However, since the relationships among all potential targets are unstructured data, it is difficult to vectorize prior information and decision-making conditions. Therefore, most of the aforementioned studies did not use deep learning methods in addressing tracking problems, which limited their ability to extract deeper and more essential information, resulting in poor adaptability.

Graph neural networks based tracking method

In deep learning, to handle non-Euclidean space data, researchers proposed Graph Neural Networks (GNNs)[30]. With the gradual development of deep learning and its applications, researchers have found that GNNs can adapt to various unstructured data. Currently, apart from the original GNNs, Graph Attention Networks (GATs) are also mainstream in the industry[14]. Some scholars have even suggested that Transformer is essentially a fully connected GNN[31]. GNNs are widely used in prediction, optimization, feature learning, and other problems due to their adaptability to unstructured data[32].

[33]employed GNNs to improve prediction results in latent spaces, while[34]used GNNs to learn feature information from unstructured data in multi-agent motion. In[35]and[36], GNNs were utilized to study team collaboration mechanisms in football and select the best combinations or players for the game[37]. applied GNNs to merge and group targets in tracking problems. GNNs can adapt to unstructured data, converting traditional methods such as traversal, screening, and matching in tracking problems into regression and classification tasks in deep learning, thereby enhancing tracking performance[38]. They have also improved the performance of tracking problems in general[39].

In the context of SOT, the application of graph attention mechanisms effectively integrates image feature information, leading to improved tracking performance[40]. For tracking problems involving objects with distinct motion patterns and characteristics, such as vehicles and high-speed particles, GNNs can process unstructured trajectory information, predict target positions, assist in target search and reconstruction, and enhance tracking performance[4142]. In multi-target tracking problems involving pedestrians and robots, GNNs primarily facilitate message passing among objects, enabling potential targets to refine their features and learn patterns from videos or contextual images[4344]. used message passing in GNNs to determine the consistency among potential targets, while[45]directly matched target features obtained from GNNs. Both methods achieved superior tracking results compared to traditional methods with the assistance of GNNs. During this process, some studies have attempted to modify the structure and frameworks of graph neural networks to enhance information aggregation capabilities and tracking efficiency within graphs[46,47]. These methods have demonstrated promising results and improvements in pedestrian tracking and matching for Multiple Object Tracking (MOT) scenarios.

Summary

Despite the abundance of detection and tracking algorithms proposed in current research, which have achieved remarkable performance in tracking feature-rich targets such as pedestrians, they cannot be directly applied to the context and problems of table tennis analysis in our study. A single detection scheme fails to provide satisfactory accuracy and recall rates, while most tracking methods primarily focus on relationship matching

issues similar to those in MOT datasets, rendering them unsuitable for our required scenarios. Although graph neural networks excel at handling unstructured data, neither GNNs nor GATs yield satisfactory results for our purposes. Notably, we recognize that the tracking tasks we envision for graph neural networks under numerous false target conditions extend beyond mere relationship matching and information fusion; they necessitate a reinterpretation (detailed in Section 3). To better accomplish this reinterpreted graph task, we propose the Graph Max-message Pass Neural Network (GMP), which we demonstrate to effectively address the challenges of table tennis tracking.

Method

Problem formulation

The input data processed by the method proposed in this paper is a video Φ, which consists of n video frames (Image, I), i.e., $\Phi = [I_1, I_2, \ldots, I_n]$. Each frame image may or may not contain a table tennis ball, and there also may be one or multiple table tennis balls in each frame image. Therefore, we assume that there are k table tennis balls ($k = 0$ *or* $k \in N^+$) in each frame image. We collect all these k table tennis balls in the tth frame image into a set B^t, where the balls are sequentially numbered as b_i^t. Specifically, b_i^t denotes the ith table tennis ball in the tth frame. It should be noted that if $k = 0$, then B^t is an empty set. In this way, we obtain a total of n sets: $B^1, B^2, \ldots, B^n$.

The continuous movement of table tennis balls in multiple frame images is called a trajectory T. Obviously, there may be multiple trajectories throughout the video, which we assume to be $T_1, T_2, \ldots, T_u, \ldots, T_s$. Any trajectory T_u represents an ordered sequence of table tennis balls $[b_{i_1}^{j_1}, b_{i_2}^{j_2}, \ldots, b_{i_m}^{j_m}]$, where $j_1 < j_2 < \ldots < j_m$. We know that $b_{i_1}^{j_1} \in B^{j_1}, b_{i_2}^{j_2} \in B^{j_2}, \ldots, b_{i_m}^{j_m} \in B^{j_m}$. It should be noted that, in $T_1, T_2, \ldots, T_s$, the table tennis balls subordinated to different trajectories T_u are all distinct. That is to say, for any given real table tennis ball b_i^t, it can only belong to a specific trajectory T_u and cannot be simultaneously associated with two trajectories. Additionally, we assume that every table tennis ball b_i^t will appear in the form of a trajectory, indicating that $T_1 \cup T_2 \cup \ldots \cup T_s = B_1 \cup B_2 \cup \ldots \cup B_n$.

Our ultimate task is to identify all the trajectories $T_1, T_2, \ldots, T_s$ within the frame images $[I_1, I_2, \ldots, I_n]$ from video Φ. An example is shown in Figure 3.

Frameworks and processes

Firstly, we will outline the entire process of the proposed method:

Step 1: Each video frame image $\{I_1, I_2, \ldots, I_n\}$ will be processed through the object detector h_{real}, resulting in a set of potential objects for each frame. Consequently, we obtain n sets of potential objects, denoted as $[O^1, O^2, \ldots, O^t, \ldots, O^n]$. These sets are similar to the sets of ping-pong balls B^t in each frame, except that B^t represents the actual table tennis targets, whereas O^t comprises potential table tennis targets. Specifically, $O^t = \{o_1^t, o_2^t, \ldots, o_k^t\}$, where o_i^t represents the i-th potential table tennis target in the t-th frame. Section 3.3 will provide a detailed introduction to the object detector h_{real}.

Step 2: Utilizing a sliding window (Window, w) and a gap (Gap, g), we sequentially select the potential target sets $O^t, O^{t+1}, \ldots, O^{t+w-1}$ obtained from Step 1 to construct a set of graphs $G = \{G_1, G_2, \ldots, G_r\}$ for further discrimination. Each graph $G : \{V, E\}$ comprises a set of nodes V and a set of edges E. The definitions of nodes and edges, as well as the specific construction method of the graphs, will be elaborated in Section 3.4.

Step 3: Each graph in the set $G = \{G_1, G_2, \ldots, G_r\}$ will be fed into the discriminator $f_{real}(\theta, G_r)$ to obtain a set of graphs $G^p = \{G_1^p, G_2^p, \ldots, G_r^p\}$, where both nodes and edges have associated probabilities. If the input graph to the discriminator is $G : \{V, E\}$, then the output probability graph is defined as $G^p : \{V^p, E^p\} \quad where\ V^p = \{(v, p(v)) | v \in V, p(v) \subset [0, 1]\}$, $E^p = \{(e, p(e)) | e \in E, p(e) \subset [0, 1]\}$. In Section 3.5, we will introduce the discriminator $f_{real}(\theta, G)$ in detail.

Step 4: Finally, based on all the probability graphs $G^p(G_1^p, G_2^p, \ldots, G_r^p)$, we derive a set of trajectories $[T_1, T_2, \ldots, T_s]$ as the final result. Section 3.6 will explain the method of obtaining the final trajectories $[T_1, T_2, \ldots, T_s]$ from the probability graphs G^p.

The whole process of the proposed method is illustrated in Figure 4.

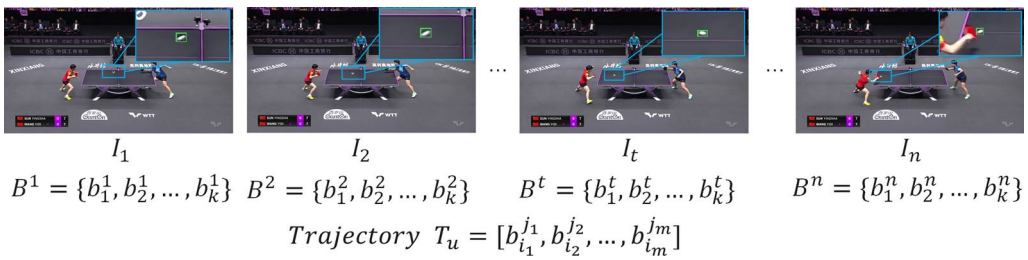


Fig. 3. Video frames and table tennis balls to be tracked.

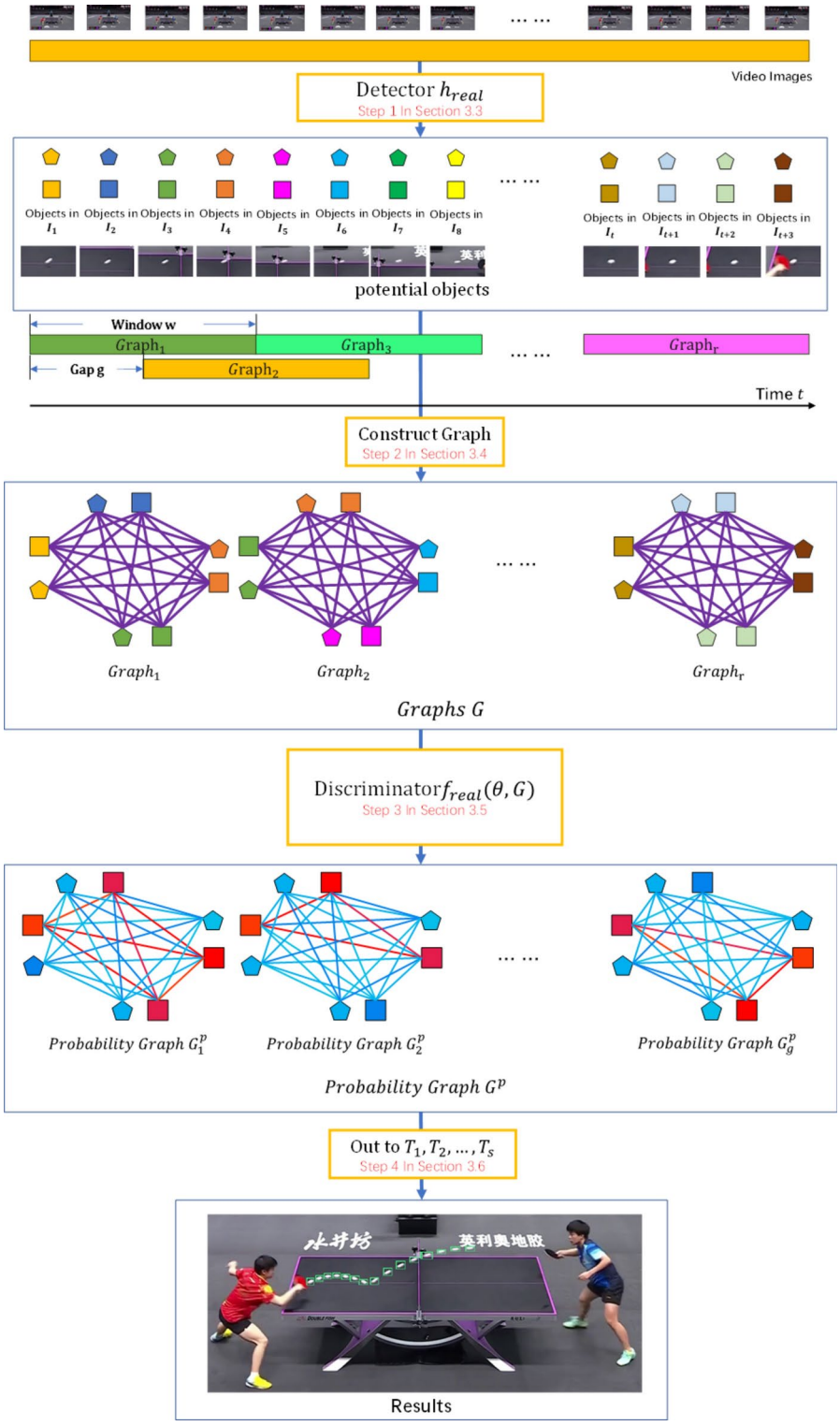


Fig. 4. Flowchart of the proposed method, Step 1 extract potential targets, Step 2 organize these potential goals into a graph, Step 3 compute the confidence of each potential target and the relationship between them (i.e., gives the confidence of the nodes and edges in the graph), Step 4 organize the results of the track based on the confidence of these goals and relationships.

In the proposed method, the two core steps are the detector and the discriminator. Below, we briefly introduce our thought of the design about these two core steps, which is important to understand following detail description of each step of the proposed method.

For the problem of trajectory tracking of a specific target in a video, the most straightforward solution is to train a detector specifically for that target. In the case of table tennis trajectory tracking, this detector is represented as $p\left(b_i^t|I_t\right)$, which detects table tennis balls from each video frame image based on the detector. Subsequently, all detected table tennis balls, denoted as b_i^j, are linked together. However, due to the sparse and easily blurred features of a single table tennis ball in the image, directly construct the detector $p\left(b_i^t|I_t\right)$ for table tennis faces numerous challenges and yields unsatisfactory results. For instance, Figure 5 demonstrates an example of the results obtained from a table tennis ball detector (YOLOv8), which is trained by the dataset mentioned in Section 4 based. It can be observed that there are numerous missed detections and false detections.

To solve this problem, we need to approach it from a different way. Since the result trajectories $[T_1, T_2, \ldots, T_s]$ to be output are independent of each other, we can simply analyze the detection process of a specific trajectory T_u. In the existing video stream $\phi = \{I_1, I_2, \ldots, I_n\}$, when we do not know T_u, we can arbitrarily designate potential targets o_i^j in different frames of the video ϕ. Subsequently, we concatenate them in the order of their appearance to obtain $T_{possible_v} = [o_{i_1}^{j_1}, o_{i_2}^{j_2}, \ldots, o_{i_m}^{j_m}]$. Evidently, we can obtain T_u sooner or later by endlessly proposing $T_{possible_v}$. Let's assume that all proposed $T_{possible_v}$ form T_Σ, i.e., $T_\Sigma = \{T_{possible_1}, T_{possible_2}, \ldots, T_{possible_v}, \ldots\}$. Now, considering the posterior probability of T_u from the perspective of T_Σ we have $p\left(T_u\middle|\phi\right) = \frac{p(T_u)p\left(\phi\middle|T_u\right)}{p(\phi)}$. Since $p(\phi)$ does not participate in the posterior calculation of T_u, according to the maximum posterior criterion, $\underset{T_u}{argmax} p\left(T_u|\phi\right) = \underset{T_u}{argmax}(\log\left(p\left(T_u\right)\right) + log(p(\phi|T_u)))\ where\ T_u \in T_\Sigma$.

Then, we delve into a detailed analysis of the above expression. Assuming that the two terms in the expression are largely independent during the optimization process (or have minimal mutual influence), we can optimize them individually:

(1) The term $\log\left(p\left(T_u\right)\right)$ can be regarded as a probabilistic model that discriminates whether T_u is sufficiently close to the prior of table tennis motion characteristics. To estimate $p\left(T_u\right)$, we need to build a discriminator based on a dataset of table tennis motions.

(2) For the second term $log(p(\phi|T_u))$, we can consider a generative model of ϕ as $\phi = \phi_{Inital} \oplus T_u$, where ϕ_{Inital} can be interpreted as the background component (or video prior) of the image stream under analysis. The relationship between these components is expressed as $p\left(\phi|T_u\right)p(T_u)p(\phi_{Inital}|\phi, T_u) = p(\phi)p(T_u|\phi_{Inital}, \phi)p(\phi_{Inital}|\phi)$. This simplifies to

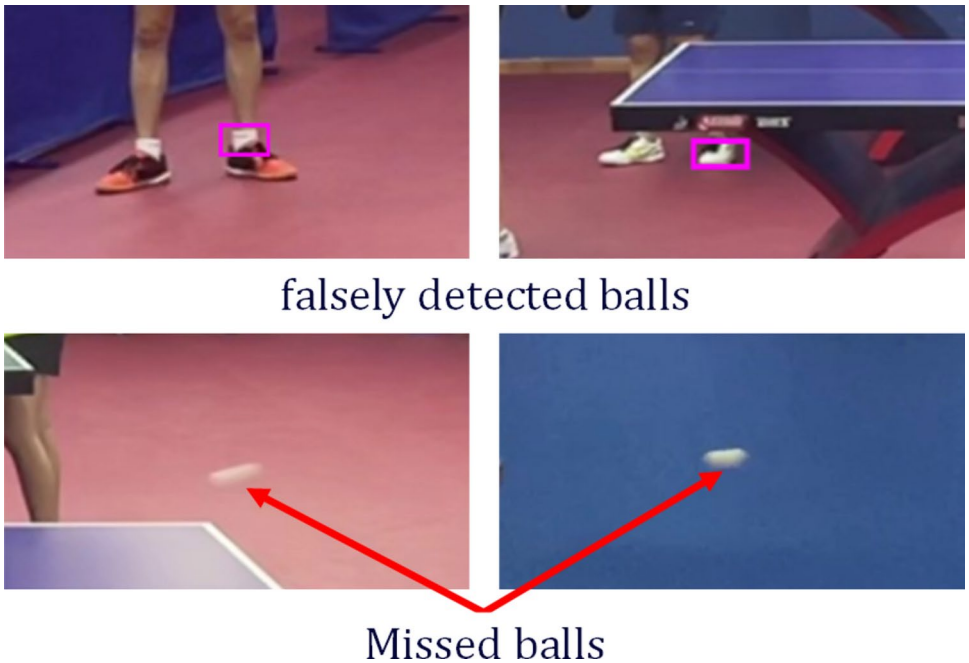


Fig. 5. Object detection results of the trained detector YOLOv8.

$p(\phi|T_u) = \frac{p(\phi)p(T_u|\phi_{Inital},\phi)p(\phi_{Inital}|\phi)}{p(T_u)p(\phi_{Inital}|\phi,T_u)}$. Given that we primarily focus on the relationship between T_u and ϕ in this generative model of ϕ, terms like $p(T_u)$ and other independent factors can be ignored, resulting in $p(\phi|T_u) \propto p(T_u|\phi_{Inital},\phi)$. Since we can estimate ϕ_{Inital} through contextual information in the video stream, $p(T_u|\phi_{Inital},\phi)$ points to a detector using the estimation of scene as a prior. That is, the proposed T_u should yield a favorable output from the detector $p(T_u|\phi_{Inital},\phi)$.Through the above derivation, we have decomposed the tracking problem into finding T_u that maximizes the relatively independent discriminator $\log(p(T_u))$ and detector $log(p\left(T_u\middle|\phi_{Inital},\phi\right))$. Let's denote the discriminator as $f(\theta,T_u)$ and the detector as $h(T_u|\mu,\phi)$. Then, $T_u = \underset{T_{possible}}{argmax}(f(\theta,T_{possible}) + h(T_{possible}|\mu,\phi))$, where θ and μ are parameters. In practical scenarios, the preliminary detector often only provides a basic reliable threshold and cannot effectively regress. Moreover, since the detector typically operates on single-frame images, we incorporate all outputs from the single-frame detector into the discriminator. In essence, our approach can be described as following formula:

$$T_u = \underset{T_{possible}}{argmax}\left(f(\theta,T_{possible})\right) \quad where\ T_{possible} = [o_{i_1}^{j_1}, o_{i_2}^{j_2}, \ldots, o_{i_m}^{j_m}]\ \ (j_1 < j_2 < \ldots < j_m),$$

$$o_i^j \in O^j\left(O^j = \left\{o_i^j \middle| h_{real}\left(o_i^j\middle|\mu, I_j\right) > threshold\right\}\right).$$

Notably, h_{real} here is still a discriminator with a prior estimation of ϕ_{Inital}, rather than a simple single-frame image detector $p(o_i^j|I_j)$. Additionally, we can implicitly learn the features of all known table tennis trajectories $T_{known}^{\Sigma} = [T_{known1}, T_{known2}, \ldots]$ through parameterized deep learning methods, obtaining the discrimination function $f(\theta,T_{possible})$, where $T_{possible}$ is the feature to be discriminated and $\theta = Train(T_{known}^{\Sigma})$ represents the parameters learned from all known table tennis trajectories T_{known}^{Σ}. Practically, due to the vast number of potential $T_{possible}$, we compose the set of potential targets $[O^1, O^2, \ldots, O^t, \ldots, O^n]$ into a graph G, and our discriminator is reformulated as $f_{real}(\theta, G)$, whose output is the previously mentioned probabilistic graph G^p.

Detector

Based on Section 3.2, we know that the detector h_{real},which detects potential targets from video frame images, originates from the probabilistic model $p(T_u|\phi_{Inital},\phi)$. This model points to a detector that uses scene estimation as a prior. Considering a frame image I_t belonging to the source video ϕ, during the derivation process, we label I_{t_Inital} as the scene for the video frame image I_t under detection. Given that the background of an image changes relatively slowly and stably compared to high-speed moving objects, adjacent frames in the temporal domain, such as $I_{t-\Delta t}, I_t, I_{t+\Delta t}$, can be considered to have the same scene $I_{t_Initial}$. Within these images, a high-speed moving object b_i^t represents the table tennis ball that we aim to detect. Assuming that the foreground images of the table tennis ball in these three frames are non-overlapping, denoted as $I_b_i^{t-\Delta t}, I_b_i^t, I_b_i^{t+\Delta t}$ (if there is overlap among $I_b_i^{t-\Delta t}, I_b_i^t, I_b_i^{t+\Delta t}$ in the video, we can increase Δt to eliminate any overlap).

Then, we have $I_{t-\Delta t} = I_{t_Inital} \oplus I_b_i^{t-\Delta t}, I_t = I_{t_Inital} \oplus I_b_i^t, I_{t+\Delta t} = I_{t_Inital} \oplus I_b_i^{t+\Delta t}$. Since our goal is to extract $I_b_i^t$, we need to perform subtraction to eliminate $I_{t_Initial}$. Calculating $\Delta^- = absdiff(I_{t-\Delta t}, I_t)$ results in $\Delta^- = I_b_i^t \oplus I_b_i^{t-\Delta t}$. Similarly, calculating $\Delta^+ = absdiff(I_{t+\Delta t}, I_t)$ gives $\Delta^+ = I_b_i^t \oplus I_b_i^{t+\Delta t}$. Finally, calculating $\Delta = absdiff(I_{t+\Delta t}, I_{t-\Delta t})$ gives $\Delta = I_b_i^{t-\Delta t} \oplus I_b_i^{t+\Delta t}$.

Considering the binarized images $\Delta_b^+, \Delta_b, \Delta_b^-$ corresbounding to $\Delta^+, \Delta, \Delta^-$, we define $M_b_i^t = (\Delta_b^+ \hat{\Delta}_b^-) \backslash (\Delta_b)$. This binarized image $M_b_i^t$ serves as the position mask for $I_b_i^t$. This method, first proposed in the[5] article, is an effective approach for detecting high-speed moving objects. Figure 6 illustrate this process.

In an ideal scenario, extracting and analyzing the connected components of the binarized image $I_b_i^t$ would yield the target set B^t at time *t*. However, due to noise in practical scenarios, $M_b_i^t$ contains numerous false targets within its connected components. We denote the set of all potential targets derived from the analysis of $M_b_i^t$ as O^t. It is certain that the set of table tennis balls we seek to find, B^t, is a subset of O^t. In reality, the number of potential targets o_i^t directly obtained from this method for each frame can be hundreds or even thousands. To minimize the number of potential targets o_i^t, we propose three methods: **noise reduction**, **filtering**, and **connected component analysis**.

Noise reduction: We notice that the modeling of $I_{t_Initial}$ is often crude, and in practical videos, due to camera motion or sensor noise, $I_{t_Initial_real} = I_{t_Initial} \oplus \sigma$, where σ represents the ghosting and noise introduced by camera movement or sensor jitter. Taking Δ^-- as an example,

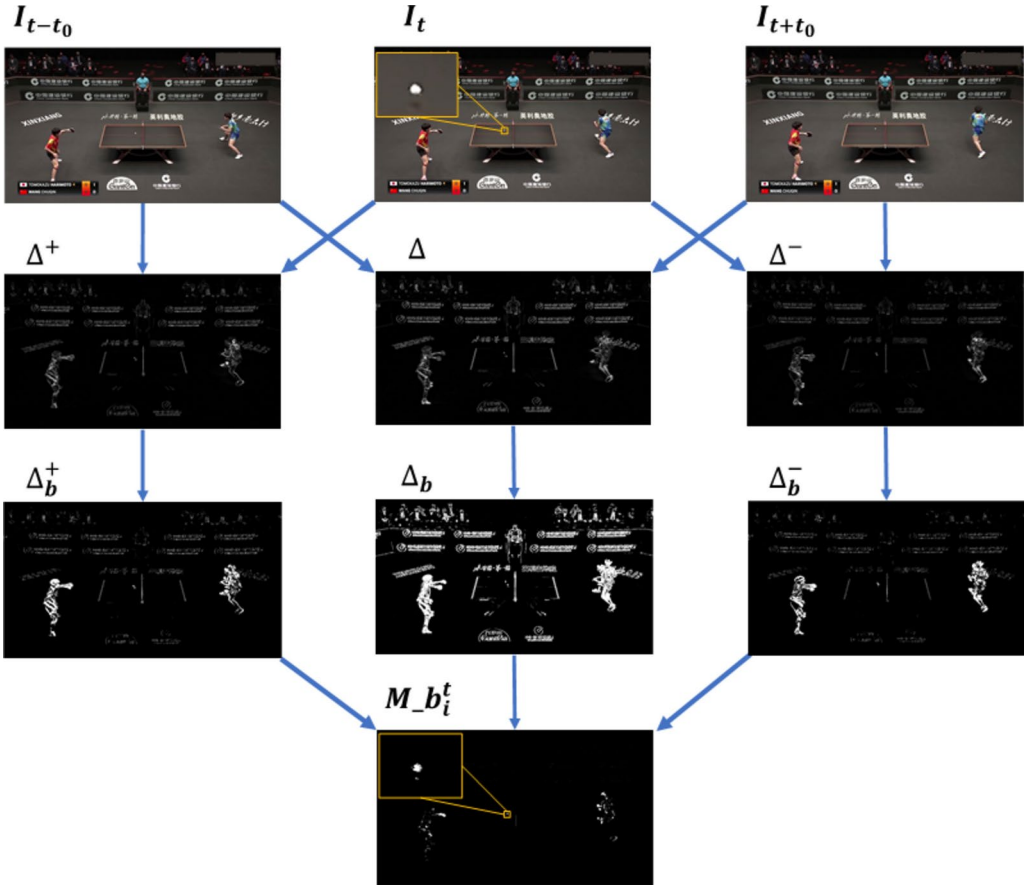


Fig. 6. The process of differential detector h_{real}.

Video Name	Stages of the method			
	Initial	After Noise reduction	After Filtering	Final
Side-Facing Male Race01	1876.08	544.51	25.02	12.65
Drive Training02	845.43	419.35	28.40	19.30

Table 1. The Average number of potential targets o_i^t in Video ϕ. We have extracted two videos from different

$\Delta^- = absdiff(I_{t-\Delta t}, I_t) = (I_b_i^t \oplus I_b_i^{t-\Delta t}) \oplus \sigma$. This can lead to the detection of excessive false o_i^t, which should originally belong to $I_{t_Initial}$. Considering that camera motion and sensor noise are typically sensitive to the edge regions of an image, we can estimate the distribution of σ across the entire image, $\sigma^{'}$, by computing the gradient of each pixel in I_t using the Sobel operator and subtracting it accordingly: $\Delta^- = (I_b_i^t \oplus I_b_i^{t-\Delta t}) \oplus \sigma \oplus (-\sigma^{'}) \approx (I_b_i^t \oplus I_b_i^{t-\Delta t})$. However, directly using I_t to compute $\sigma^{'}$ would result in the edges of $I_b_i^t$ being eroded into $\sigma^{'}$, leading to missed detections. Therefore, when actually computing $\sigma^{'}$, we use the images $I_{t-\Delta t}$ and $I_{t+\Delta t}$ instead.

Filtering: Although much noise is removed during the noise reduction step, there are still numerous isolated noise points present in $M_b_i^t$. The table tennis balls b_i^t that we aim to extract typically belong to a larger connected component. Therefore, we can filter out these isolated noise points by applying an "erosion" followed by a "dilation" process, while barely altering the connected component where b_i^t resides. Through this approach, we can significantly reduce the number of connected components and potential targets o_i^t, greatly reducing the computational burden without compromising the final output accuracy.

Connected Component Analysis: To further reduce the number of o_i^t, we can utilize some prior knowledge about the table tennis balls b_i^t to conduct a simple analysis and filtering of o_i^t. For instance, we can predefine the



Fig. 7. An example of all the potential targets o_i^t from I_t.

shape and color characteristics of the table tennis balls. By analyzing the shape, size of the connected component where o_i^t resides, as well as the color of the corresponding region in I_t, we can further filter out a portion of o_i^t.

The above method does not improve the effect of the method significantly, but it can greatly increase the speed of the method by reducing the number of false targets (Table 1 gives two examples). Figure 7 is an example to show all the potential targets o_i^t on an image. Up to this point, we do not have sufficient information to distinguish whether these potential targets o_i^t are part of the final trajectory T_u. The subsequent steps will involve screening and stitching them together to form the final output trajectory T_u.

Construct graph

After obtaining the potential targets $[O^1, O^2, \ldots, O^t, \ldots, O^n]$ throughout the entire video, we sequentially select consecutive frame sets based on the window length, w, and stride, g, which is specified in Section 3.2. The range of set are: $[0,\ w-1],\ [g,\ g+w-1],\ \ldots,\ [k_1 \cdot g,\ k_1 \cdot g + w - 1],\ \ldots,\ [k * g,\ n]$, and the total number of such sets is $\left\lfloor \frac{n-w}{g} \right\rfloor + 1$. Using the detector described in Section 3.3, we obtain the target sets corresponding to these consecutive frames, $[O^t, O^{t+1}, \ldots, O^{t+w-1}]$. We consider all targets o_i^j belonging to this set as nodes and connect all nodes belonging to different target sets O^{t_i} as edges. The resulting graph can be described as $G : \{V, E\}\ \ where\ V = \{v_x | potential\ target\ o_{ix}^{jx}\}, E = \{e_{xy} = (o_{ix}^{jx}, o_{iy}^{jy}) | o_{ix}^{jx}, o_{iy}^{jy}, j_x \neq j_y\}$.

After obtaining all the potential targets o_i^t, a straightforward approach would be to iterate through all possible $T_{possible}$ and then select the tracking result T_u using a learned discriminator $f(\theta, T_{possible})$. Researchers in the[6] have proposed similar ideas. The study revealed that directly iterating through all possible $T_{possible}$ without any filtering is completely unrealistic due to the astronomical number of potential combinations. However, it is straightforward to observe that every $T_{possible} = [o_{i1}^{j1}, o_{i2}^{j2}, \ldots, o_{im}^{jm}]$ can be summarized by two essential elements: the target o_{ix}^{jx} and the relationship between two targets o_{ix}^{jx} and o_{iy}^{jy}.

After the construction of graph G, any potential trajectory in $T_{possible}$ can be uniquely mapped to a subgraph $G_{T_{possible}} : \{V_{T_{possible}}, E_{T_{possible}}\}\ where\ V_{T_{possible}} = \{v_x | o_{ix}^{jx} \in T_{possible}\}, E_{T_{possible}} = \{e_{xy} = (o_{ix}^{jx}, o_{iy}^{jy}) | o_{ix}^{jx}, o_{iy}^{jy} \in T_{possible}\}$. By arranging the nodes according to the frame order of the targets and adding edges connecting them, $T_{possible}$ corresponds to a path in the graph, $[v_1, e_{12}, v_2, e_{23}, v_3, \ldots, v_x, e_{xy}, v_y, \ldots, v_m]$, where the sequence of nodes represents the trajectory we aim to output. Notably, if $e_{xy} = (o_{ix}^{jx}, o_{iy}^{jy}) \in T_{possible}$, then $j_x \neq j_y$, indicating that two potential targets do not belong to the same frame. This is the reason why we require $j_x \neq j_y$ when constructing the edges of graph G. Furthermore, if the distance between o_{ix}^{jx} and o_{iy}^{jy} in the image plane or the difference in their sizes is excessively large, based on priors and preset conditions, the existence of the edge e_{xy} connecting o_{ix}^{jx} and o_{iy}^{jy} becomes unnecessary. These priors and preset conditions significantly reduce the complexity of graph G. Since node v_x corresponds to a target o_{ix}^{jx} that belongs to only one of the sets $[O^t, O^{t+1}, \ldots, O^{t+w-1}]$, specifically O^{t_j}, and all nodes $o_i^{t_j}$ belonging to the same set O^{t_j} are not connected to each other. Therefore, graph G is an N-partite graph, where N = w (window length).

Discriminator

Tasks of discriminator

Inspired by the applications of graph neural networks discussed in Chapter 2, we employ a graph neural network to implement the discriminator $f_{real}(\theta, G)$. We design the following task for it: Suppose the result trajectory T_u have a path in graph G, represented as $\left[v_{u_1}, e_{u_1u_2}, v_{u_2}, e_{u_2u_3}, v_{u_1}, \ldots, v_{u_x}, e_{u_xu_y}, v_{u_y}, \ldots, v_{u_m}\right]$. The function $f_{real}(\theta, G)$ maps graph

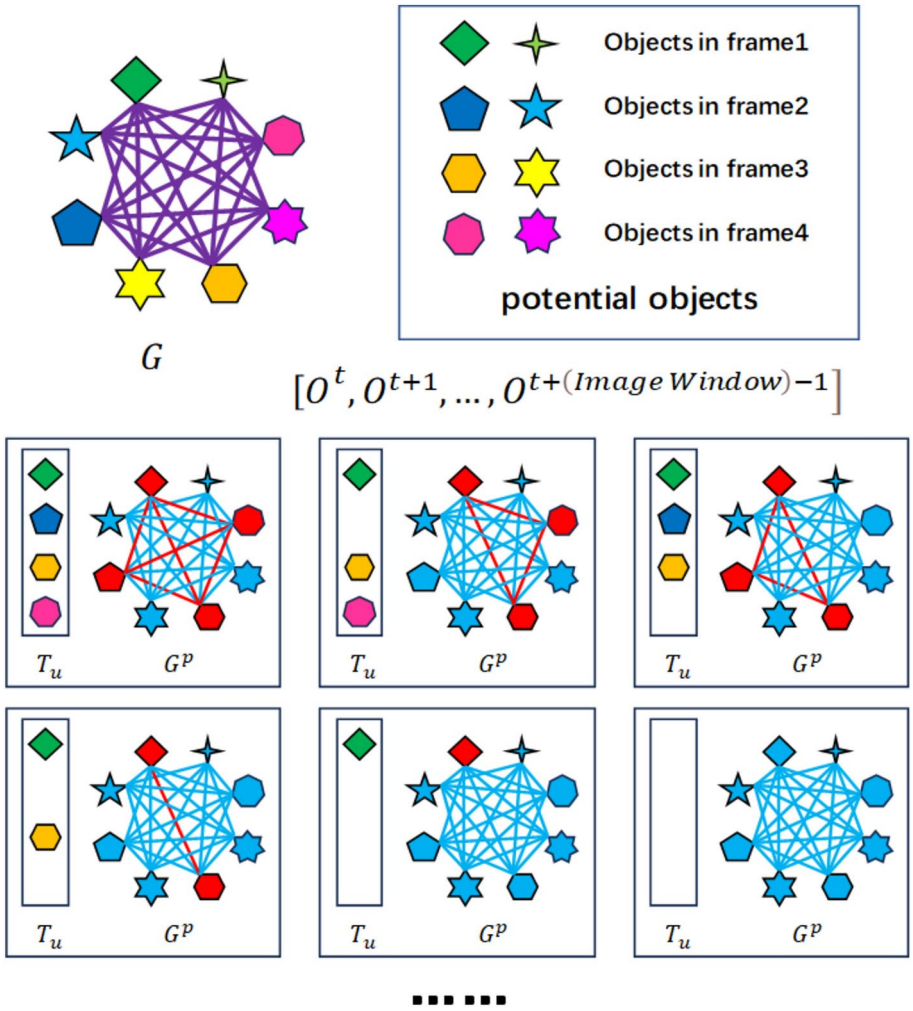


Fig. 8. The correspondence between trajectory T_u and the probabilistic graph G^p (Red color represents nodes *v* and edges *e* with $p(v) = 1$ or $p(e) = 1$, while blue color represents nodes and edges with $p(v) = 0$ or $p(e) = 0$.). Those nodes and edges marked red are part of the trajectory, while the blue nodes and edges are

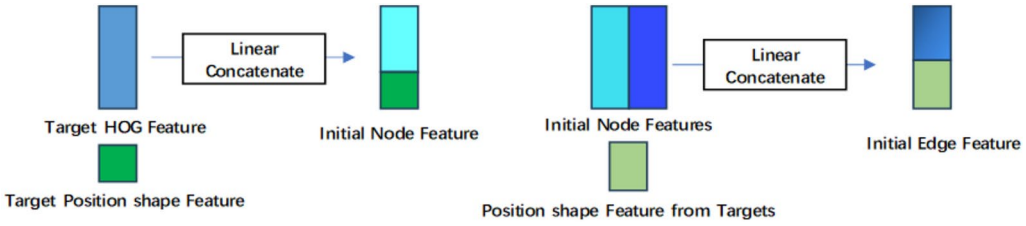


Fig. 9. The computation process for the initial features p^0 of a node *v*.

G to a probabilistic graph G^p, where each node *v* and edge *e* has an associated probability. Specifically, $G^p : \{V', E'\}$ *where* $V' = \{(v, p(v)) | v \in V, p(v) \subset [0, 1]\}$, $E' = \{(e, p(e)) | e \in E, p(e) \subset [0, 1]\}$ *with* $V, E \in G$. The task requires that for all *v* or *e*, if *v* or *e* belongs to T_u, then in the probabilistic graph G^p, $p(v) = 1$ or $p(e) = 1$; otherwise, $p(v) = 0$ or $p(e) = 0$. It is noteworthy that if there are multiple paths between two nodes that are not completely overlapping and a node $v_{u_x} \in T_u$ exists on these paths, such as $[v_{u_1}, e_{u_1u_2}, v_{u_2}, e_{u_2u_3}, v_{u_3}]$ and $[v_{u_1}, e_{u_1u_3}, v_{u_3}]$, then all nodes *v* and edges *e* on these paths should have $p(v) = 1$ and $p(e) = 1$ in the probabilistic graph G^p. Figure 8 illustrates the relationship between the trajectory

T_u and the probabilistic graph G^p using an example of four consecutive image frames (with a window length w=4).

Structure of discriminator

In Section 3.4, we mentioned that the discriminator of our algorithm will be a graph neural network, denoted as $f_{real}(\theta, G)$. Prior to this, we need to compute the initial features for both nodes *v* and edges *e*. The features of a node *v* are composed of the local image HOG features, position, and shape information corresponding to its underlying object o_i^t. Due to the sparsity of HOG features, a full-connect layer (also called Linear Layer in pytorch) is applied to compress the features before inputting them into the graph neural network. The initial features of an edge $e_{xy} = (o_{i_x}^{j_x}, o_{i_y}^{j_y})$ are jointly determined by the position and shape information, as well as the initial features of $o_{i_x}^{j_x}$ and $o_{i_y}^{j_y}$ (Figure 9 illustrate this process).

After initializing the node features p^0 and edge features q^0, we pass them through a custom three-layer graph neural network, GMP (Graph Max-message Pass Neural Network). The steps and details of each layer of this model will be explained in detail later. The GMP graph neural network performs feature propagation and embedding to obtain the final node features $p^{'}$ and edge features $q^{'}$. Subsequently, we design two multi-layer perceptrons, w_1 and w_2, to score the final feature $p^{'}$ and $q^{'}$ of each node and edge, respectively. This allows us to obtain $p(v)$ and $p(e)$ in the probabilistic graph G^p. The specific process is illustrated in Figure 10.

GMP layer

Like all graph neural networks (GNNs), the Graph Message Passing (GMP) network also relies on message passing to update node and edge feature vectors. However, unlike other GNNs, the GMP network operates as follows: Firstly, each node v_x in the GMP network collects all messages *Mes* from its neighbors and concatenates them into a "message tensor" Mes_{Ax}. Secondly, it scores the edges e_{xy} connected to neighboring nodes based on the neighbors' messages *Mes*, yielding gra_{xy}. Thirdly, it applies max-pooling to the "message tensor" Mes_{Ax} to obtain a node-specific message Mes_x, which is used to update the node features. Finally, it updates the edge features based on the feature vector of the edge e_{xy}, the feature vectors of its adjacent nodes v_x,v_y, and the scores gra_{xy}, gra_{yx} assigned by nodes v_x and v_y to the edge.

The four main steps outlined above correspond to the four primary operations *f, l, g, h* in each layer of GMP, each containing learned parameter sets $\theta_1, \theta_2, \theta_3, \theta_4$. Detailed steps are elaborated below, which readers may skip if not seeking specifics. Assuming the input data consists of previous or initial node and edge features $p^n \in R^i$, $q^n \in R^j$, the "message tensor" $Mes \in R^k$, updated node features $p^{n+1} \in R^{ii}$, and edge features $q^{n+1} \in R^{jj}$:

The predefined inputs and outputs for f,l,g,h can be defined as follows:

$f(x, \theta_1)$: Takes features $In \in R^{2i+j}$ where $In = (p_x^n, q_{xy}^n, p_y^n)$ as input and produces a message $Mes \in R^k$ as output.

$l(x, \theta_2)$: Takes a message $Mes \in R^k$ as input and outputs a real-valued score $gra \in R$.

$g(x, \theta_3)$: Takes features $(Mes, p_x^n) \in R^{k+i}$ as input and produces new node features $p_x^{n+1} \in R^{ii}$ as output.

$h(x, \theta_4)$: Takes a tuple $(p_x^{n+1}, gra_{xy}, p_y^{n+1}, gra_{yx}, q_{xy}^n)$ as input and outputs new edge features $q_{xy}^{n+1} \in R^{jj}$.

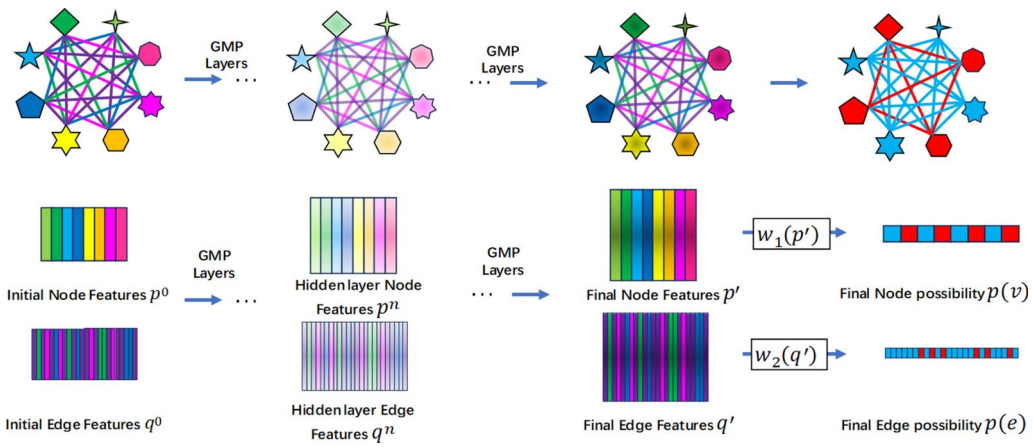


Fig. 10. The structure and computation process of the discriminator, with the computation layer by layer, the characteristics of each node and edge are constantly changing, until the confidence level is finally obtained by

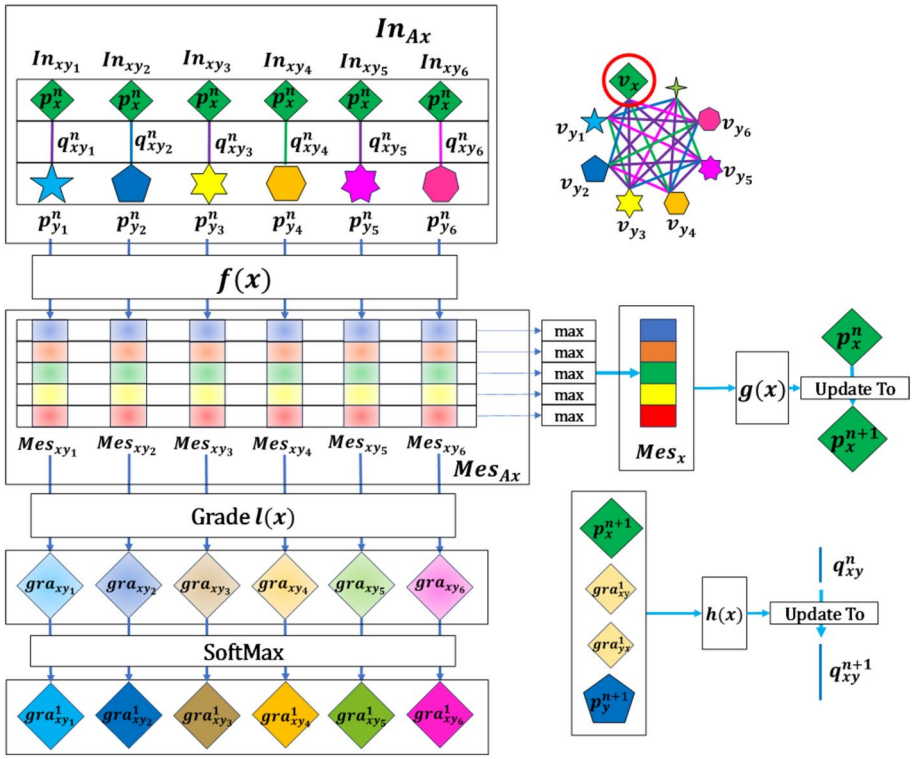


Fig. 11. The feature updating process in each layer of the GMP. We illustrate the feature update process for node v_x and edge e_{xy}.

Considering the feature update processes for nodes and edges separately, GMP primarily comprises two parts, Step A and Step B, focused on updating node features pn and edge features qn, respectively (The whole update process is illustrate in Figure 11).

Step A: Calculate the "message tensor" *Mes*, the "message passing strength" *gra*, and update all node features. *for* each node $v_x \in V$:

1. Iterate over all adjacent nodes $v_{y_1}, v_{y_2}, \ldots, v_{y_g}, \ldots v_{y_h}$ and their corresponding edges $e_{xy_1}, e_{xy_2}, \ldots, e_{xy_g}, \ldots, e_{xy_h} \in E$, each iterate get a triplet, all of them can be noted as $(v_x, v_{y_1}, e_{xy_1}), (v_x, v_{y_2}, e_{xy_2}), \ldots, (v_x, v_{y_g}, e_{xy_g}), \ldots, (v_x, v_{y_h}, e_{xy_h})$. Then, concatenate their corresponding feature vectors $\left(p_x^n, q_{xy_1}^n, p_{y_1}^n\right), \left(p_x^n, q_{xy_2}^n, p_{y_2}^n\right), \ldots, (p_x^n, q_{xy_g}^n, p_{y_g}^n), \ldots, \left(p_x^n, q_{xy_h}^n, p_{y_h}^n\right)$. Each iteration yields results $In_{xy_1}, In_{xy_2}, \ldots, In_{xy_g}, \ldots In_{xy_h}$.
2. Concatenate all In_{xy_g} to obtain the tensor $Mes_{Ax} \in R^{m \times k}$ (using 0^k for non-adjacent node tensors).
3. $Mes_x = max\,(Mes_{Ax}, dim = 0)$ to obtain the "message" for node v_x.
4. $gra_{xy_g} = l\left(Mes_{xy_g}, \theta_2\right), gra_{xy_g} \in R$. Pass $Mes_{xy_1}, Mes_{xy_2}, \ldots, Mes_{xy_g}, \ldots, Mes_{xy_h}$ through $l(x, \theta_2)$ to obtain $gra_{xy_1}, gra_{xy_2}, \ldots, gra_{xy_g}, \ldots, gra_{xy_h}$, which are the "message" weights for each triplet (v_x, v_{y_g}, e_{xy_g}). Associate these weights with the respective edges e_{xy_g}, where edge e_{xy_g} receives gra_{xy_g} (the "message" weight obtained through node v_{y_g}) is $gra_{y_g x}$).
5. $p_x^{n+1} = g((Mes_x, p_x^n), \theta_3)$ to update the feature vector for node v_x.**Step B:** Update edge features based on the "message passing strength" *gra*.

1. After completing Step A, each edge e_{xy_g} possesses "message" weights $e_{xy_g}(gra_{xy_g}, gra_{y_g x})$. Each node v_x finds all its adjacent edges $e_{xy_1}, e_{xy_2}, \ldots, e_{xy_g}, \ldots, e_{xy_h}$. As mentioned earlier, graph *G* exhibits an n-partite structure due to the window length w (n = window length), and adjacent nodes $v_{y_1}, v_{y_2}, \ldots, v_{y_g}, \ldots v_{y_h}$ belong to n-1 parts among them. Assume without loss of generality that $v_{y_g^1}, v_{y_g^2}, \ldots, v_{y_g^n}$ are the same node located in the same part, and the adjacent edges connecting to the same part, $e_{xy_g^1}, e_{xy_g^2}, \ldots, e_{xy_g^n}$, have their respective $e_{xy_g}(gra_{xy_g}, gra_{y_g x})$. Perform a SoftMax mapping on $[gra_{xy_g^1}, gra_{xy_g^2}, \ldots, gra_{xy_g^n}]$ to obtain $[gra_{xy_g^1}^1, gra_{xy_g^2}^1, \ldots, gra_{xy_g^n}^1]$. For each edge e_{xy}, after completing the above steps, we obtain thresholded values that regress to (0,1), namely $(gra_{xy_g}^1, gra_{y_g x}^1)$.
2. *for* each edge $e_{xy} \in E$ in the graph:

$q_{xy}^{n+1} = h((p_x^{n+1}, gra_{xy}^1, p_y^{n+1}, gra_{yx}^1, q_{xy}^n), \theta_4)$ to update the feature vector of edge e_{xy}.

Message Vectors and Node features Update

Here, we focus on explaining the design rationale for the feature *p* component of our discriminator related to node *v*. In essence, within the designed tracking task framework, table tennis nodes belonging to the same trajectory should be able to identify each other through a message-passing process. While updating their features based on these "messages," they should actively "reject" messages from other nodes. Consequently, we have devised a message-passing framework as illustrated in Figure 11 (characterized by the use of max-pooling to process all messages received by the nodes). This section provides a detailed mathematical explanation. Readers may skip this part if they are not seeking in-depth details.

Returning to the graph task we set out in Section 3.4, if node v_{u_x} belongs to the graph path $\left[v_{u_1}, e_{u_1u_2}, v_{u_2}, e_{u_2u_3}, v_{u_1}, \ldots, v_{u_x}, e_{u_xu_y}, v_{u_y}, \ldots, v_{u_m}\right]$ corresponding to trajectory T_u, let's assume under ideal conditions that the node and edge features belonging to trajectory T_u are $[p_{u_1}^{'}, q_{u_1u_2}^{'}, p_{u_2}^{'}, q_{u_2u_3}^{'}, p_{u_3}^{'}, \ldots, p_{u_x}^{'}, q_{u_xu_y}^{'}, p_{u_y}^{'}, \ldots, p_{u_m}^{'}]$. The output of classifier w_1 is $p(T_u|p_{u_1}^{'}), p(T_u|p_{u_2}^{'}), \ldots, p(T_u|p_{u_m}^{'})$; and the output of classifier w_2 is $p(T_u|q_{u_1u_2}^{'}), p(T_u|q_{u_2u_3}^{'}), \ldots, p(T_u|q_{u_{m-1}u_m}^{'})$. This implies that after message passing through the GMP network, the features $p_{u_x}^{'}$ and $q_{u_xu_y}^{'}$ exhibit consistency within the feature space and are highly relevant to T_u. Let's assume that T_u corresponds to the core features $p_u^{'}$ and $q_u^{'}$ in the feature space. Then, the final output features $p_{u_x}^{'}$ and $q_{u_xu_y}^{'}$ of the node v_{u_x} and edge $e_{u_xu_y}$ belonging to T_u should be as close as possible to $p_u^{'}$ and $q_u^{'}$. Simultaneously, the output features $p_x^{'}$ and $q_{xy}^{'}$ of other nodes v_x and edges e_{xy} should be as far away as possible from $p_u^{'}$ and $q_u^{'}$. Revisiting graph *G*, if we desire specific node features p_x^n and edge features q_{xy}^n along a certain path, targeted message passing between nodes is necessary. Nodes v_{ux} belonging to the path corresponding to T_u engage in message passing with other nodes v_{u_y} also belonging to T_u, mutually recognizing their membership in T_u and estimating $p_u^{'}$ based on messages Mes_{ux} from neighboring nodes. This allows for consistency correction and, after multiple iterations of message passing, results in $p_{u_x}^{'}$.

Due to the presence of numerous false targets in graph *G*, node v_{u_x} belonging to T_u should ideally only receive messages from nodes v_{u_y} that also belong to T_u. We observe that since table tennis exhibits specific motion *pattern* across multiple video frames, the triplets $(v_{u_x}, e_{u_xu_y}, v_{u_y})$ between nodes v_{u_x} and v_{u_y} in T_u should also possess these motion *pattern*. Therefore, our previously designed function $f(x, \theta_1)$ must possess the ability to recognize these *pattern*. Let's design $f(x, \theta_1)$ such that the higher the input *x* aligns with the characteristic *pattern*, the higher the output function value. Considering that graph *G* contains a significant number of false target nodes *v*, node v_{ux} should filter them out. As a result, the message it receives from neighboring nodes is $Mes_{ux} = \arg\max_{y_g} f((v_x, v_{y_g}, e_{xy_g}), \theta_1)$. Notably, the motion of table tennis may exhibit multiple patterns $pattern_k$, thus $Mes_{ux} \in R^k$.

For any given node v_x, the message Mes_x received informs it of the specific patterns $pattern_k$ exhibited by its neighboring nodes. Node v_x can then use Mes_x to modify its node feature p_x^n and record the details of this message passing, resulting in p_x^{n+1}. That is, $p_x^{n+1} = g((Mes_x, p_x^n), \theta_3)$. Figure 12 illustrates that our algorithm gradually distinguish different classes of nodes through feature update processes. Figure 13 illustrates the detail step of $g(x, \theta_3)$ and $h(x, \theta_4)$, where $h(x, \theta_4)$ will be shown in Section 3.5.5.

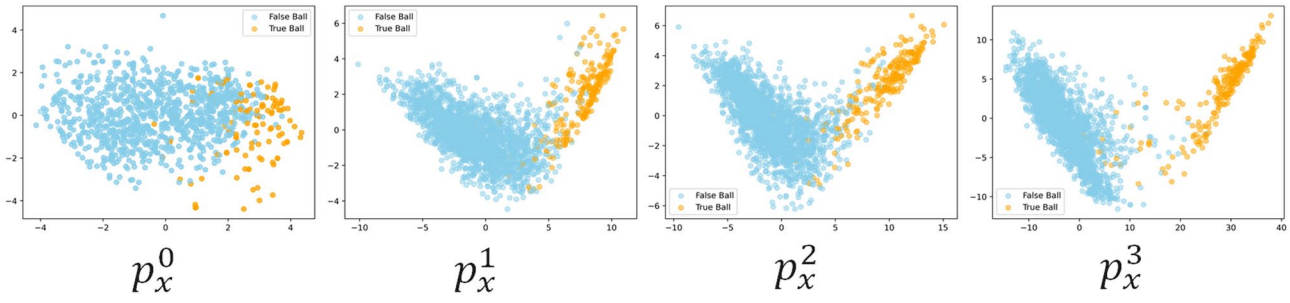


Fig. 12. The PCA clustering results of node features at different levels demonstrate that, as message passing and feature updating proceed, the difference in features between positive samples (orange) and negative samples (blue) becomes increasingly significant.

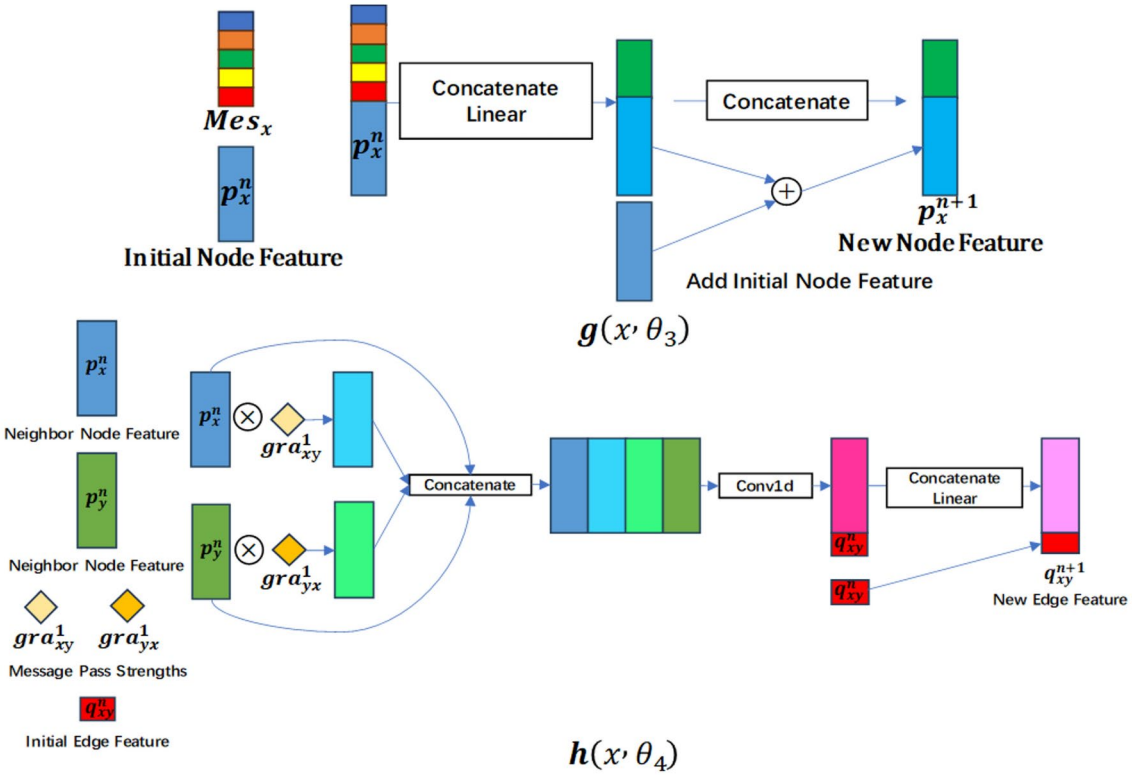


Fig. 13. Schematic Diagram of the Node and Edge Update Functions $g(x, \theta_4)$ and $h(x, \theta_4)$.

Edge features update and grading edge with SoftMax

In updating edge features, we observe that in graph neural networks, edges serve as the carriers of message passing, and their properties are primarily determined by the nodes they connect. Furthermore, within the designed task framework, there exists a mutual exclusion relationship among some of the edges connected to the same node. Based on this observation, we employ softMax to score the edges and utilize these scores for updating edge features. This section primarily elaborates on the design rationale and provides a mathematical explanation. Readers who are not seeking detailed derivations may skip this part.

Similarly, if an edge $e_{u_x u_y}$ belongs to the graph path $\left[v_{u_1}, e_{u_1 u_2}, v_{u_2}, e_{u_2 u_3}, v_{u_1}, \ldots, v_{u_x}, e_{u_x u_y}, v_{u_y}, \ldots, v_{u_m}\right]$ corresponding to T_u, then the triplet $(v_{u_x}, e_{u_x u_y}, v_{u_y})$ composed of the edge $e_{u_x u_y}$ and its two endpoint nodes v_{u_x} and v_{u_y} should possess a motion *pattern*. Furthermore, if $e_{u_x u_y}$ belongs to the graph path corresponding to T_u, then v_{u_x} and v_{u_y} should exchange messages through $e_{u_x u_y}$ to bring their respective features $p^{'}_{u_x}$ and $p^{'}_{u_y}$ closer to the feature vector $p^{'}_{u}$ corresponding to T_u.

Then, a straightforward approach would be to determine the feature q_{xy}^{n+1} of the edge e_{xy} based on the neighbor node features p_x^{n+1} and p_y^{n+1} after the message passing process, i.e., $q_{xy}^{n+1} = h(p_x^{n+1}, p_y^{n+1}, \theta_4)$. However, if our goal is to achieve the targeted and specific message passing mentioned earlier, this simple approach falls short as it treats the edge e_{xy} as an appendage to the nodes v_x and v_y, ignoring the relationships between edges in the graph structure.

Given the n-partite structure of graph G and the absence of nodes belonging to the same part in the graph path $\left[v_{u_1}, e_{u_1 u_2}, v_{u_2}, e_{u_2 u_3}, v_{u_1}, \ldots, v_{u_x}, e_{u_x u_y}, v_{u_y}, \ldots, v_{u_m}\right]$ corresponding to T_u, there exist numerous mutual exclusion relationships among the edges incident to a given node v_x, such as $e_{xy_1}, e_{xy_2}, \ldots, e_{xy_g}, \ldots, e_{xy_h}$. For instance, if nodes $v_{y_g^1}$ and $v_{y_g^2}$, both neighbors of node v_x, belong to the same part, then the edges connecting them, $e_{xy_g^1}$ and $e_{xy_g^2}$, exhibit a mutual exclusion relationship.

Assuming that the node v_{u_x} belonging to T_u has neighboring nodes $v_{u_y}, v_{y_g^1}, v_{y_g^2}, \ldots, v_{y_g^n}$ from the same part, where only v_{u_y} belongs to T_u, we cannot distinguish based on the available information whether $v_{u_y}, v_{y_g^1}, v_{y_g^2}, \ldots, v_{y_g^n}$ belong to T_u. Since we know that only one of them belongs to T_u , it follows that only one of the edges connecting them belongs to T_u. This leads to the constraint $p(T_u|e_{u_x u_y}) + p(T_u|e_{u_x y_g^1}) + p(T_u|e_{u_x y_g^2}) + \ldots + p(T_u|e_{u_x y_g^n}) \leq 1$.

We implement the constraint and the probability $p(T_u|e_{xy})$ using SoftMax and a scoring function $l(x, \theta_2)$, respectively. Consequently, the edge e_{xy} receives two weights, gra_{xy}^1 and gra_{yx}^1, which we refer to as "message

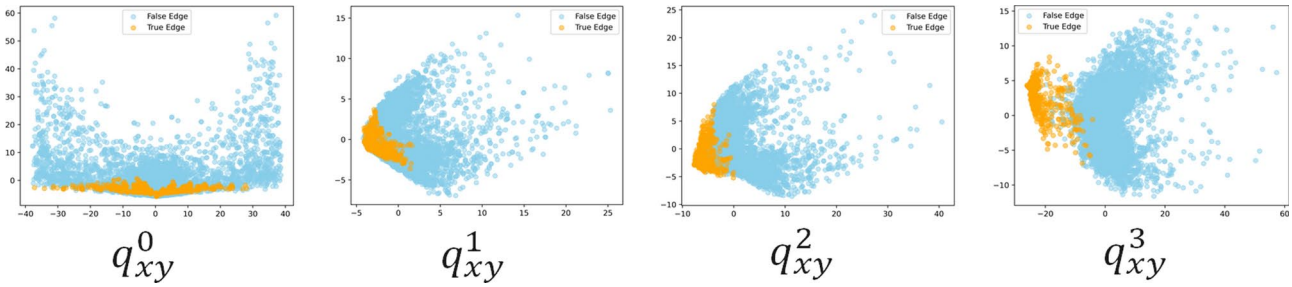


Fig. 14. The PCA clustering results of edge features at various levels indicate that, as message passing and feature updating progress, the differences in features between positive samples (orange) and negative samples (blue) become increasingly pronounced.

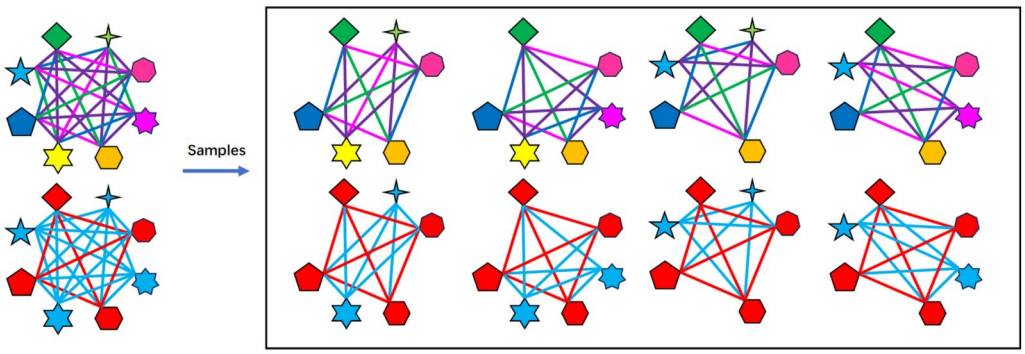


Fig. 15. Schematic Diagram of the Graph Sampling Process. Red represents the Positive Sample while Blue represents Negative one. (Note: We Retain All Positive Samples and Only Sample on Negative Samples).

passing strengths". Based on these weights and the updated node features p_x^{n+1} and p_y^{n+1} after message passing, we update the feature of edge e_{xy} as $q_{xy}^{n+1} = h((p_x^{n+1}, gra_{xy}^1, p_y^{n+1}, gra_{yx}^1, q_{xy}^n), \theta_4)$.

If $e_{xy} \in T_u$, p_x^{n+1} and p_y^{n+1} provide specific information about T_u to the edge e_{xy}, and through $h(x, \theta_4)$, $q_{xy}^{'}$ is steered towards $q_n^{'}$. Conversely, if e_{xy} does not belong to T_u, $h(x, \theta_4)$ uses the tuple $(p_x^{n+1}, gra_{xy}^1, p_y^{n+1}, gra_{yx}^1, q_{xy}^n)$ to determine that $e_{xy} \notin T_u$ and steers $q_{xy}^{'}$ away from $q_n^{'}$.

Alternatively, if a significant amount of "message passing" occurs between nodes v_x and v_y through e_{xy}, resulting in p_x^{n+1} and p_y^{n+1}, the simplest approach is to record this information in q_{xy}^{n+1}. In this case, $h(x, \theta_4)$ needs to utilize the tuple $(p_x^{n+1}, gra_{xy}^1, p_y^{n+1}, gra_{yx}^1, q_{xy}^n)$ to make the necessary judgments and maintain an accurate "record". Figure 14 illustrates that our algorithm gradually distinguish different classes of edge through process above.

Training method and loss function

According to the graph tasks we have previously defined, the multi-layer perceptron, w_1 and w_2, perform supervised learning on the outputs $p(v)$ and $p(e)$ based on the final features $p^{'}$ and $q^{'}$ of nodes v and edges e, respectively, comparing them with the pre-defined labels $p_{GT}(v)$ and $p_{GT}(e)$. However, during the training process, the large number of false targets o_i^t in the original video often results in a large-scale graph G. Taking the scenario of Image Window=8 as an example, a single graph in the Ground Truth dataset for training may contain up to hundreds of nodes and tens of thousands of edges, leading to significant resource consumption that precludes training on most GPUs.

Drawing from our discussions in Sections 3.5.3 and 3.5.4, message passing in the GMP model is typically targeted, with negative samples or messages passed between nodes belonging to different categories often being less significant. Therefore, we can randomly sample negative samples or nodes from other categories while preserving positive samples or a specific category of nodes[48]. A large-scale graph G can be sampled to generate multiple small-scale graphs, and combined with randomization and the setting of hyperparameters such as batch size during training, the training process becomes more stable, often leading to more ideal results. Additionally, we can re-sample the graph G between training epochs to obtain different outcomes, effectively implementing a form of data augmentation that can effectively mitigate overfitting in the model. An example of sampling is shown in Figure 15.

Interestingly, our graph neural network does not consume excessive memory during the inference mode due to the elimination of the need to store gradients, computation graphs, and intermediate results. Consequently, the algorithm does not require sampling the graph *G* during inference.

In terms of loss function selection, we have chosen the cross-entropy loss function. For the graph $G\{V, E\}$, we compute the average loss across all nodes and edges and then directly sum them up:

$$loss = -\frac{1}{V(G)}\sum \left(p_{GT}(v_x)\log\left(p(v_x)\right) + \left(1 - p_{GT}(v_x)\right) log\left(1 - p(v_x)\right)\right)$$
$$+ \frac{1}{E(G)}\sum \left(p_{GT}(e_{xy})\log\left(p(e_{xy})\right) + \left(1 - p_{GT}(e_{xy})\right) log\left(1 - p(e_{xy})\right)\right)$$
$$where\ v_x \in V,\ e_{xy} \in E;\ V, E \in G\{V, E\}$$

Now let us revisit the cross-entropy loss function, which is expressed as $l = -\sum ylogx + (1-y)log(1-x)$ where *y* represents the label of the sample and *x* is the output result. Assuming in practical scenarios that the average correct prediction rate for positive samples is *p* and for negative samples is *q*, with the respective numbers of positive and negative samples being *m* and *n*, the total loss can be expressed as $l = -\frac{1}{m+n}(mlogp + n\log q)$.

We can calculate the derivatives of the loss function with respect to *p* and *q* as $\frac{\partial l}{\partial p} = -\frac{1}{p}\frac{m}{m+n}$, $\frac{\partial l}{\partial q} = -\frac{1}{q}\frac{n}{m+n}$. Since *p* and *q* are generated by the Sigmoid function, the gradients passed back to the classifier are $\frac{\partial l}{\partial w} = \frac{\partial l}{\partial p}\frac{\partial p}{\partial w} + \frac{\partial l}{\partial q}\frac{\partial q}{\partial w} = -\frac{m}{m+n}\frac{1}{p}p(1-p) - \frac{n}{m+n}\frac{1}{q}q(1-q) = -\frac{1}{m+n}(m(1-p) + n(1-q))$. Here, the gradient ratio contributed by *p* and *q*, $\frac{\nabla p}{\nabla q}$, is $\frac{m(1-p)}{n(1-q)}$.

Furthermore, we can compute precision (*pre*) and recall as $pre = \frac{\frac{m}{m+n}p}{\frac{n}{m+n}(1-q)+\frac{m}{m+n}p} = \frac{pm}{pm+(1-q)n}$ and $recall = \frac{mp}{m} = p$. When $pre = recall$, we have $\frac{m(1-p)}{n(1-q)} = 1$ and $\nabla p = \nabla q$, and the model evenly improves the performance of both positive and negative samples. When $pre > recall$, $\frac{m(1-p)}{n(1-q)} < 1$, $\nabla p < \nabla q$, and the model tends to improve the performance of negative samples more, emphasizing *recall*. Conversely, when $pre < recall$, $\frac{m(1-p)}{n(1-q)} > 1$, $\nabla p > \nabla q$, and the model favors improving the performance of positive samples, emphasizing *pre*.

The original cross-entropy function aims to achieve $pre = recall$. However, when $m \ll n$, we have $1 - p \gg 1 - q$, meaning the model tends to improve *q* to a higher level before improving *p*. This can result in a low precision *pre* for a significant portion of the training process, and positive samples may not receive sufficient attention.

To address this, we consider a weighted loss function: $l = -\sum a_1 y\log x + a_2(1-y)log(1-x)$ where a_1 and a_2 are weighting coefficients. When $a_1 = a_2 = 1$, it becomes the original cross-entropy function. Setting $a_1 > a_2$ allows the model to focus more on positive samples, emphasizing the improvement of *p* and recall. Conversely, $a_2 > a_1$ directs the model's attention to negative samples, emphasizing the enhancement of *q* and *pre*.

At the initial stage of our model training, we set $a_1 > a_2$. As the loss function decreases and *p* and *q* improve, we gradually decrease a_1 until $a_1 \approx a_2 \approx 1$. In practice, we use the following expressions for a_1 and a_2: $a_1 = \frac{m+n}{2Max((loss+0.15)\sqrt{mn}+(-loss+0.85)(\frac{m+n}{2}),\sqrt{mn})}$ and $a_2 = \frac{m+n}{2(m+n-Max((loss+0.15)\sqrt{mn}+(-loss+0.85)(\frac{m+n}{2}),\sqrt{mn}))}$, In this expression, *loss* refers to the value of loss function from the previous epoch, and the initial *loss* can be considered as positive infinity or a large number.

Organize and output results

In Section 3.4, we constructed several graphs using the potential targets o_i^j from the set $[O^1, O^2, \ldots, O^t, \ldots, O^n]$. For each probability graph G^p, we obtained the probabilities $p(v_x)$ and $p(e_{xy})$ that each node v_x and edge e_{xy} belongs to a particular trajectory T_u in the graph *G*. These probabilities correspond to the likelihood of a potential target $o_{i_x}^{j_x}$ belonging to a trajectory T_u, denoted as $p(o_{i_x}^{j_x})$, and the probability of two potential targets $o_{i_x}^{j_x}$ and $o_{i_y}^{j_y}$ belonging to the same trajectory, denoted as $q(o_{i_x}^{j_x}, o_{i_y}^{j_y})$. It is worth noting that during the process of graph construction based on Image Window and Gap, some targets $o_{i_x}^{j_x}$ and connections $(o_{i_x}^{j_x}, o_{i_y}^{j_y})$ may appear in multiple probability graphs G^p. In such cases, we simply select the maximum values of $p(o_{i_x}^{j_x})$ and $q(o_{i_x}^{j_x}, o_{i_y}^{j_y})$ as the output of the Discriminator.

Subsequently, we utilize the output results $p(o_{i_x}^{j_x})$ and $q(o_{i_x}^{j_x}, o_{i_y}^{j_y})$ to derive the resulting trajectories $T_1, T_2, \ldots, T_s$. The specific process is outlined as follows:

Step 1.

Set threshold values $p_{\min}$ and $q_{\min}$ for $p(o_{i_x}^{j_x})$ and $q(o_{i_x}^{j_x}, o_{i_y}^{j_y})$, respectively (where $p_{min}, q_{min} > 0$).**Step 2.**

Filter out all potential targets $o_{i_x}^{j_x}$ with $p\left(o_{i_x}^{j_x}\right) < p_{\min}$.**Step 3.**

For each remaining potential target $o_{i_x}^{j_x}$ with $p\left(o_{i_x}^{j_x}\right) \geq p_{\min}$, we establish a trajectory T_x containing only that particular target, i.e., $T_x = [o_{i_x}^{j_x}]$. The set T is composed of all such trajectories T_x, $T = \{T_1, T_2, \ldots, T_x, \ldots, T_y, \ldots\}$.**Step 4.**

Identify the two most suitable trajectories T_x and T_y for merging in T.

Initial $q_{temp} = 0, temp_x = -1, temp_y = -1$;
For (T_x, T_y) in T (where $x \neq y$):

Where $T_x = [o_{i_{x_1}}^{j_{x_1}}, o_{i_{x_2}}^{j_{x_2}}, \ldots, o_{i_{x_n}}^{j_{x_n}}], T_y = [o_{i_{y_1}}^{j_{y_1}}, o_{i_{y_2}}^{j_{y_2}}, \ldots, o_{i_{y_m}}^{j_{y_m}}]$
Initial $count = 0$, $q_{result} = 0$
For $(o_{i_{x_k}}^{j_{x_k}}, o_{i_{y_l}}^{j_{y_l}})$ in (T_x, T_y):

If any $j_{x_k} = j_{y_l}$:

$q_{result} = 0$

Break

Elif $q(o_{i_{x_k}}^{j_{x_k}}, o_{i_{y_l}}^{j_{y_l}})$ doesn't exist:

Continue

Else:

$q_{result} \mathrel{+}= q(o_{i_{x_k}}^{j_{x_k}}, o_{i_{y_l}}^{j_{y_l}})$

$count++$

End For

If *count* != 0:

$q_{result} = \frac{q_{result}}{count}$

If $q_{result} > q_{temp}$:$q_{temp} = q_{result}, temp_x = x, temp_y = y$;

Else:

Continue

End For

Goto Step 5.**Step 5.**

If $q_{temp} > q_{\min}$:

1. Merge T_{temp_x} and T_{temp_y} into T_{new} by combining all potential targets from T_{temp_x} and T_{temp_y} into a single sequence and arranging them in ascending order based on the values of $j_{x_1}, j_{x_2}, \ldots, j_{x_n}$ and $j_{y_1}, j_{y_2}, \ldots, j_{y_m}$;
2. Remove T_{temp_x} and T_{temp_y} from T and add T_{new} to T (update T);
3. Then, **Return to** Step 4.

Else:

Goto Step 6.**Step 6.**

For the set $T = \{T_1, T_2, \ldots, T_s\}$, filter out all trajectories with a length of 1, i.e., remove all T_u from T where $T_u = [o_{i_u}^{j_u}]$.**Finish Process**

The final set $T = \{T_1, T_2, \ldots, T_s\}$ obtained at the end of the algorithm represents the final output result.

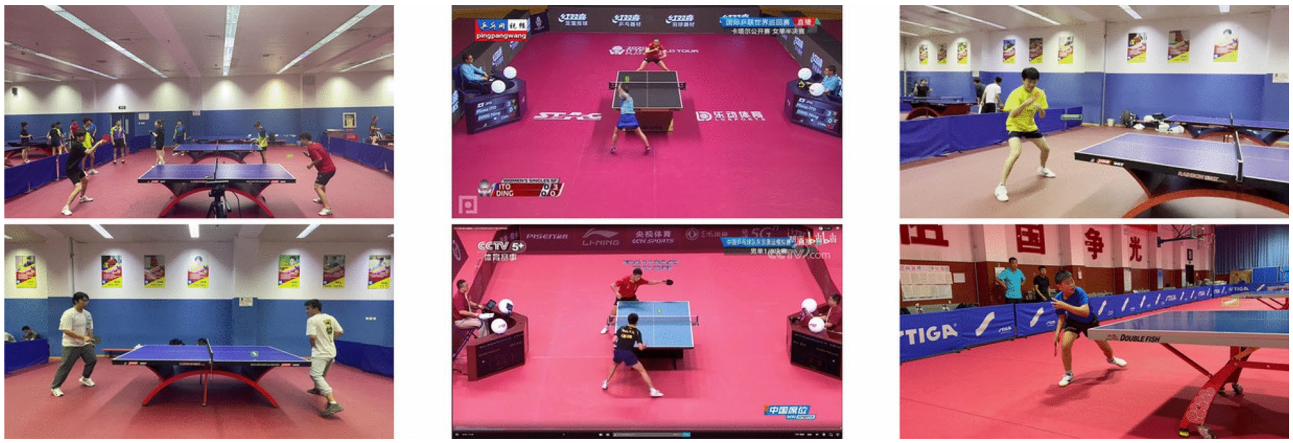


Fig. 16. Illustrations of Images from Different Perspectives: Side-Facing (Left), Directly-Facing (Middle), Player-Facing (Right).

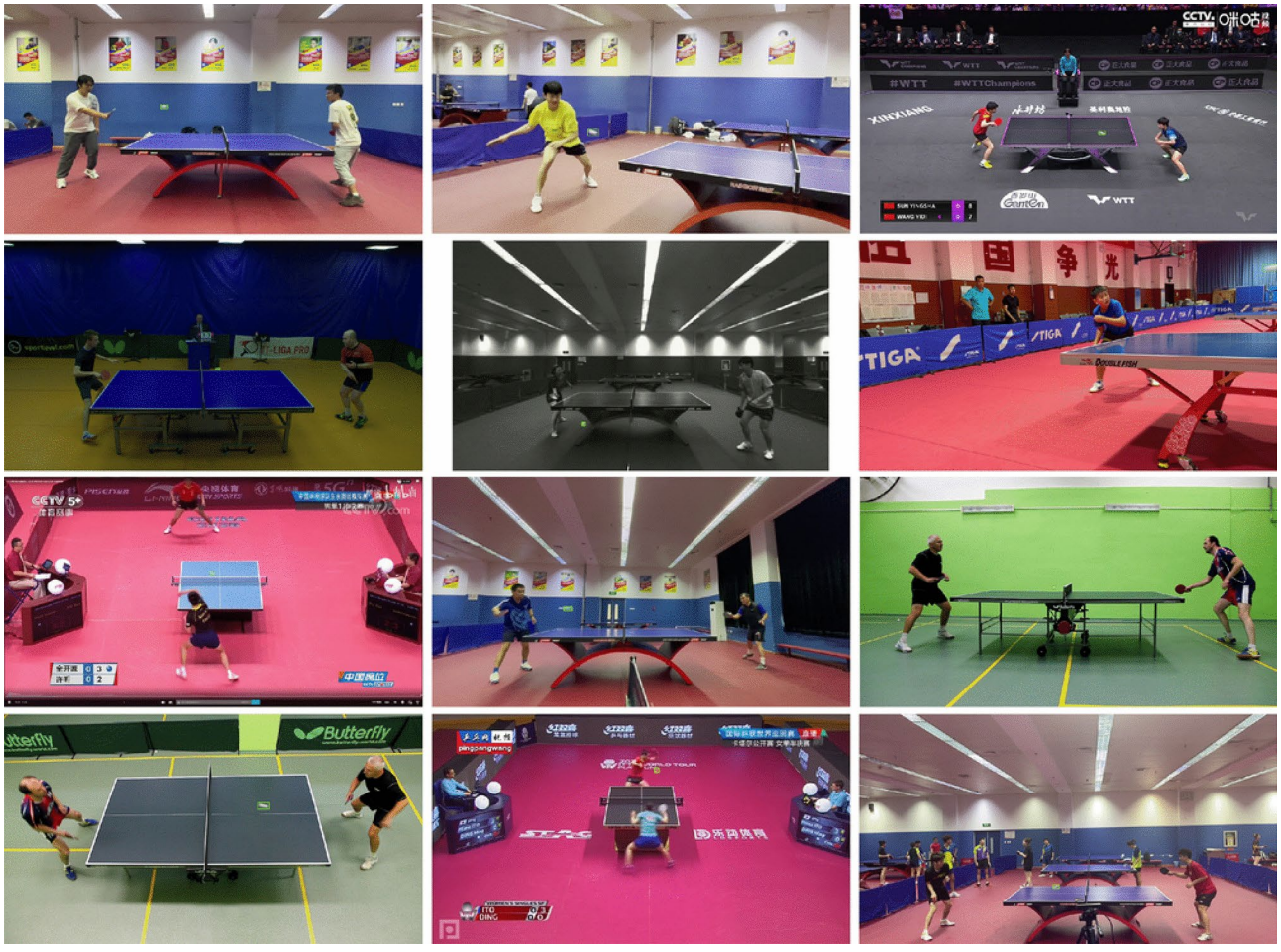


Fig. 17. Original Video Images from the Dataset.

Evaluation and results

To train a useful model and validate effectiveness of the algorithm in a real-world, it is imperative to collect table tennis match videos that closely resemble the intended usage scenarios. These videos must be annotated according to the requirements of tactical analysis, serving as the dataset for algorithm training and evaluation. Additionally, to verify the algorithm's effectiveness and advancement, a comparison with similar solutions is necessary. Such comparisons can also assist in identifying any issues within the algorithm.

Datasets

We have augmented the[67] dataset with a significant number of videos, resulting in a comprehensive collection encompassing 20 independent videos as the dataset. Each video is a continuous recording under a fixed scenario, totaling 13,950 frames and 6,027 annotated table tennis balls. In selecting and annotating the videos, we took into account their diversity: our dataset includes scenarios such as competition, training, and warm-up; players ranging from amateurs, student athletes, to professional athletes; and various camera perspectives including side-facing (viewing the table tennis table from the side), directly-facing (viewing the table tennis table from the back), and player-facing (viewing the player directly). In subsequent text, we will refer to the side-facing, directly-facing, and player-facing perspectives as Side-Facing, Directly-Facing, and Player-Facing, respectively. Scenes captured from different perspectives are illustrated in Figure 16. Over 45% of the dataset comes from videos recorded on mobile phones, while more than 30% originates from competition replays sourced from online and televised broadcasts on platforms such as Migu video and CCTV. Additionally, we have appropriately referenced some open-source datasets[59] to provide horizontal comparisons and references. An example frame from the dataset is shown in Figure 17, and the specific details of the videos in the dataset are summarized in Table A1 of Appendix A.

Results and comparison

Training and results

We selected the first 17 out of the 20 videos from the dataset for training and testing purposes, while reserving the last 3 videos for validation. It is worth noting that while the last 3 videos share similar scenarios, only one of the 17 videos used for training and testing exhibits a comparable setting. This approach was adopted because the perspectives and scenarios of the last 3 videos are the most popular among current table tennis broadcasts and tactical analyses, and are also the most likely scenarios for the application of our algorithm. Given the diversity of video scenarios and the inconsistency in the number of video frames and table tennis balls captured from different perspectives, we took extra care in dividing the training and testing sets to minimize the impact of data imbalance. For some videos, we chose the first 70% of consecutive frames as the training set, while for others, we selected the first 50%. The remaining 50% of consecutive frames from each video were designated as the testing set. (Specific ranges are detailed in Table A2. of Appendix A.)

During training, we employed the sampling scheme mentioned in Section 3.5.6 to sample the graph G constructed from the training set, while no sampling was performed on the testing set. The final counts of nodes v and edges e in the entire training set are presented in Table 2.

For training, we set the learning rates for the three-layer message-passing graph neural network GMP and the classifiers w_1, w_2 to 0.005 and 0.002, respectively. In analyzing the results, we simply used a threshold of 0.5 to categorize nodes v and edges e into positive and negative samples. The performance criteria are precision, recall, and F1-score, which are calculated by TP/(TP + FP), TP/(TP + FN), and 2TP/(2TP + FN + FP), respectively. Here, TP, FP, and FN stand for the number of true positives, false positives, and false negatives, respectively.

The variation of the loss function (Section 3.5.5) and classification accuracy during the entire training process are depicted in Figure 18. Training was conducted using two NVIDIA Tesla-T4 GPUs. After sampling the training set, each epoch comprised approximately 1000 steps, resulting in rapid model convergence. The optimal model was obtained at epoch=8. Additionally, we re-sampled the training set every two training rounds, as described in Section 3.5.6. The initial learning rate for the GMP component was set to 5×10^{-4} with a minimum decay of 2×10^{-4}, while the initial learning rates for the classifiers w_1 and w_2 were set to 2×10^{-4} with a minimum decay of 1×10^{-4}.

Experimental results demonstrate the stability of our training approach, with good performance after model convergence. Only minor overfitting and jittering phenomena were observed.

Recalling to the problem definition in Section 3.1, we need to compare the differences between the set of trajectories $\{T_1, T_2, \ldots, T_s\}$ output by the algorithm and the Ground Truth set $T_1^{GT}, T_2^{GT}, \ldots, T_s^{GT}$. For the i-th table tennis ball b_i^t in the t-th frame belonging to the Ground Truth set, if b_i^t is also present in the output trajectory set $\{T_1, T_2, \ldots, T_s$, it will be labeled as a true positive sample. Conversely, b_i^t will be labeled as a false negative sample if it is not in the output set. Similarly, any sample belonging to the output trajectory

Attribute	Positive Num	Negative Num	Total Num
Edge in Train Set	41,750	5,278,699	5,320,449
Node in Train Set	14,694	190,358	205,052
Edge in Test Set	15,455	13,048,646	13,064,101
Node in Test Set	5,572	233,227	238,799

Table 2. Number of Nodes v and Edges e in the Test Set and Training Set.

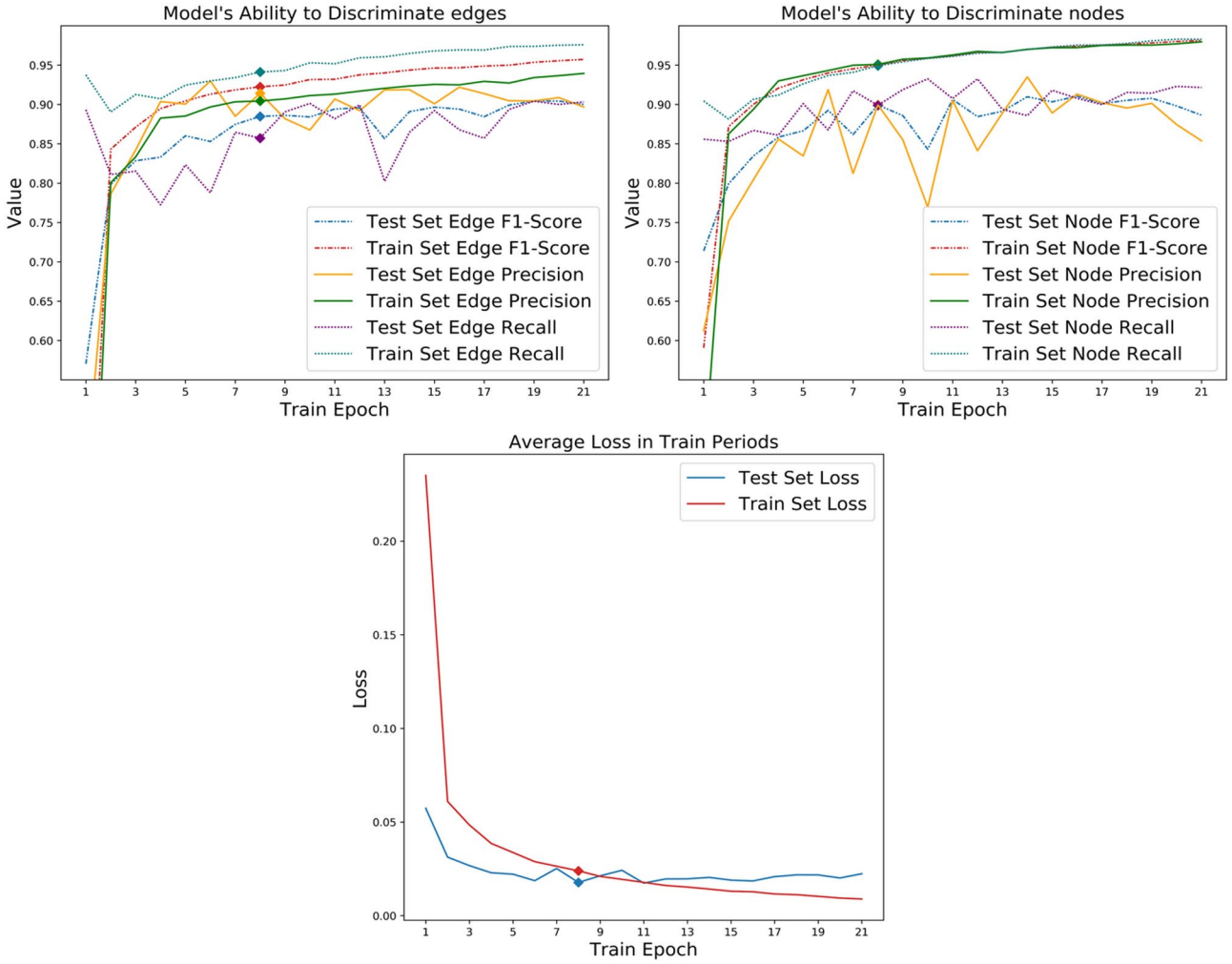


Fig. 18. The variation curves of loss function and model performance during training.

set $\{T_1, T_2, \ldots, T_s\}$ but not to the Ground Truth set $T_1^{GT}, T_2^{GT}, \ldots, T_s^{GT}$ will be labeled as a false positive sample. We will calculate the precision, recall, and F1-score for table tennis ball targets (Target) based on the number of these samples.

Apart from analyzing the table tennis balls b_i^t, we also need to assess the trajectories T_u. Consider the Ground Truth trajectory $T_u^{GT} = \left[b_{i_1^{GT}}^{j_1^{GT}}, b_{i_2^{GT}}^{j_2^{GT}}, \ldots, b_{i_m^{GT}}^{j_m^{GT}}\right]$, where all adjacent table tennis balls $(b_{i_x^{GT}}^{j_x^{GT}}, b_{i_y^{GT}}^{j_y^{GT}})$ with $(\ j_1^{GT} < j_2^{GT} < \ldots < j_x^{GT} < j_y^{GT} < \ldots < j_m^{GT})$ are identified. In total, $m-1$ pairs of adjacent table tennis balls $(b_{i_x^{GT}}^{j_x^{GT}}, b_{i_y^{GT}}^{j_y^{GT}})$ can be extracted from T_u^{GT}, denoted as the relationship $R(b_{i_x^{GT}}^{j_x^{GT}}, b_{i_y^{GT}}^{j_y^{GT}})$. Next, we investigate whether the pair $(b_{i_x^{GT}}^{j_x^{GT}}, b_{i_y^{GT}}^{j_y^{GT}})$ belongs to the same trajectory T_u within the output trajectory set $\{T_1, T_2, \ldots, T_s\}$. If the pair is indeed present in the same trajectory T_u, it is labeled as a true positive sample. Otherwise, $R(b_{i_x^{GT}}^{j_x^{GT}}, b_{i_y^{GT}}^{j_y^{GT}})$ is labeled as a false negative sample.

Finally, we examine all relationships $R(b_{i_x}^{j_x}, b_{i_y}^{j_y})$ within the output trajectory set $\{T_1, T_2, \ldots, T_s\}$. If $b_{i_x}^{j_x}$ and $b_{i_y}^{j_y}$ do not belong to the same trajectory in the Ground Truth set $\{T_1^{GT}, T_2^{GT}, \ldots, T_s^{GT}\}$, then $R(b_{i_x}^{j_x}, b_{i_y}^{j_y})$ is labeled as a false positive sample. Similarly, we calculate the precision, recall, and F1-score for all relationship groups $R(b_{i_x}^{j_x}, b_{i_y}^{j_y})$ (Relationship) based on the sample counts. This analysis provides insights into the effectiveness of the algorithm in reconstructing table tennis ball trajectories.

Based on the content of Section 3.6, the manually set thresholds $p_{\min}$ and $q_{\min}$ can influence the final test results. To assess this impact, we varied $p_{\min}$ within the range of [0.05, 0.7] with a step size of 0.05. For each setting of $p_{\min}$, we ran the algorithm with three different values of $q_{\min}$: $p_{\min}$, $p_{\min} - 0.05$, and $p_{\min} + 0.05$.

	Target									Relationship								
	Ave (0.05–0.70)			Ave (0.30–0.70)			Ave (0.40–0.60)			Ave (0.05–0.70)			Ave (0.30–0.70)			Ave (0.40–0.60)		
VideoName	Pre.	Rec.	F1.	Pre.	Rec.	F1.	Pre.	Rec.	F1.	Pre.	Rec.	F1.	Pre.	Rec.	F1.	Pre.	Rec.	F1.
Directly-Facing Female Race01	0.63	0.72	0.67	0.75	0.71	0.73	0.76	0.72	0.74	0.58	0.69	0.63	0.69	0.69	0.69	0.71	0.70	0.70
Directly-Facing Female Race02	0.86	0.92	0.89	0.93	0.91	0.92	0.95	0.93	0.94	0.78	0.91	0.84	0.86	0.89	0.88	0.88	0.91	0.89
Directly-Facing Female Race03	0.71	0.92	0.80	0.82	0.90	0.86	0.84	0.91	0.88	0.61	0.86	0.72	0.70	0.83	0.76	0.72	0.84	0.77
Directly-Facing Female Race04	0.84	0.80	0.82	0.93	0.77	0.84	0.94	0.76	0.84	0.69	0.71	0.70	0.78	0.68	0.73	0.94	0.76	0.84
Directly-Facing Female Race05	0.72	0.87	0.79	0.85	0.84	0.85	0.86	0.84	0.85	0.71	0.84	0.77	0.84	0.82	0.83	0.85	0.82	0.83
Warm-up 01	0.83	0.96	0.89	0.87	0.95	0.91	0.87	0.95	0.91	0.79	0.92	0.85	0.83	0.90	0.87	0.84	0.91	0.87
Student Race 01 (partly)	0.93	0.94	0.94	0.95	0.94	0.94	0.95	0.94	0.94	0.87	0.92	0.89	0.89	0.92	0.90	0.95	0.94	0.94
Warm-up 02	0.83	0.99	0.90	0.85	0.99	0.91	0.85	0.99	0.91	0.83	0.98	0.90	0.85	0.98	0.91	0.85	0.98	0.91
openTT Race (partly)	0.86	0.98	0.92	0.88	0.98	0.93	0.88	0.98	0.93	0.85	0.97	0.91	0.87	0.97	0.92	0.87	0.97	0.92
Attack Training (partly)	0.78	0.91	0.84	0.81	0.90	0.85	0.81	0.90	0.85	0.80	0.89	0.84	0.82	0.88	0.85	0.82	0.88	0.85
Drive Training01	0.70	0.96	0.81	0.77	0.96	0.86	0.78	0.96	0.86	0.72	0.96	0.82	0.79	0.95	0.87	0.80	0.96	0.87
Drive Training02	0.95	0.95	0.95	0.97	0.94	0.96	0.97	0.94	0.96	0.92	0.94	0.93	0.94	0.93	0.94	0.95	0.93	0.94
Side-Facing Female Race01	0.93	0.96	0.94	0.94	0.95	0.95	0.95	0.96	0.95	0.90	0.92	0.91	0.92	0.91	0.92	0.93	0.91	0.92
Directly-Facing Male Race01	0.98	0.97	0.98	0.99	0.97	0.98	0.99	0.97	0.98	0.92	0.96	0.94	0.93	0.95	0.94	0.93	0.96	0.94
FMO paint	0.95	0.63	0.75	0.96	0.51	0.67	0.95	0.51	0.67	0.85	0.51	0.64	0.81	0.37	0.51	0.78	0.36	0.49
FMO side	0.90	0.95	0.92	0.93	0.94	0.93	0.93	0.95	0.94	0.83	0.93	0.88	0.89	0.93	0.91	0.86	0.94	0.90
FMO top	0.86	0.97	0.91	0.90	0.96	0.93	0.90	0.96	0.93	0.86	0.97	0.91	0.84	0.95	0.89	0.87	0.94	0.91
Total Train & Test	0.84	0.95	0.89	0.88	0.94	0.91	0.88	0.95	0.91	0.83	0.93	0.88	0.86	0.92	0.89	0.87	0.93	0.90
Side-Facing Female Race02	0.94	0.92	0.93	0.95	0.90	0.93	0.95	0.90	0.93	0.88	0.91	0.89	0.88	0.89	0.89	0.88	0.89	0.89
Side-Facing Female Race03	0.97	0.93	0.95	0.98	0.93	0.95	0.99	0.93	0.96	0.89	0.92	0.91	0.90	0.91	0.91	0.91	0.92	0.91
Side-Facing Male Race01	0.98	0.99	0.98	0.99	0.98	0.99	0.99	0.99	0.99	0.92	0.99	0.95	0.93	0.98	0.95	0.93	0.98	0.96
Total Validation	0.97	0.95	0.96	0.98	0.95	0.96	0.98	0.95	0.96	0.90	0.95	0.92	0.91	0.94	0.92	0.91	0.94	0.93

Table 3. Experimental Results of the Algorithm on our datasets. In this table, Ave(0.05–0.70) refers to the mean average precision obtained with the threshold $p_{\min}$ ranging from 0.05 to 0.70, and the same applies to other projects. Pre., Rec., and F1. are abbreviations for Precision, Recall, and F1-Score, respectively. The first 17 out of the 20 videos in the table are used as Train & Test dataset, while the last 3 videos are used as validation set.

By averaging the output results, we obtained the mean average precision , mean average recall, and mean average F1-score.

Furthermore, we calculated the average values of these metrics across different threshold ranges. When annotating the results, we used parentheses to indicate the threshold ranges, such as (0.40–0.60) and (0.30–0.70). These ranges represent the $p_{\min}$ values tested within the specified intervals with a step size of 0.05, and the results were averaged accordingly.

The final results on our dataset are presented in Table 3. The data indicate that our proposed algorithm is not highly sensitive to the manually set thresholds $p_{\min}$ and $q_{\min}$, and it achieves good performance on most datasets.

Compare with other methods

To validate the performance and advancement of our algorithm, we examined other object tracking algorithms and potential alternative solutions, and compared them one by one. The results indicated that our algorithm excels in overall performance.

Comparison with Directly Applying Object Detection Algorithms (YOLOv8)[12]: We filtered out all images containing table tennis balls from the training and testing sets, and converted the annotated table tennis ball trajectories $\{T_1^{GT}, T_2^{GT}, \ldots, T_s^{GT}\}$ into the dataset format required by YOLOv8. Subsequently, we divided the YOLOv8 dataset into training and testing sets in the ratio of 85% and 15%, respectively. We then trained the default YOLOv8n model for 100 epochs to obtain the best-performing model. In the data evaluation, if the confidence score of a candidate bounding box inferred by the YOLOv8 model exceeded a certain threshold (conf > threshold) and its IOU(Intersection over Union) with any original annotation box of a table tennis ball in the dataset was greater than 0.4, the target was labeled as True Positive. Conversely, it was labeled as False Negative. All candidate boxes with confidence scores above the threshold (conf > threshold) but with IOU values less than 0.4 with all original annotation boxes of table tennis balls in the dataset were labeled as False Positive.

Comparison with Tracking Algorithms: SOT (Step Mode)[13]: In SOT algorithms, target tracking in videos relies on manual annotations in the first frame. However, the videos in our dataset contain numerous segments without table tennis balls, making it inappropriate to directly apply the SOT algorithm to accurately assess its tracking capability. Therefore, we disassemble the trajectories $T_u^{GT} = \left[b_{i_1^{GT}}^{j_1^{GT}}, b_{i_2^{GT}}^{j_2^{GT}}, \ldots, b_{i_m^{GT}}^{j_m^{GT}}\right]$ in the dataset into relationships $R(b_{i_x^{GT}}^{j_x^{GT}}, b_{i_y^{GT}}^{j_y^{GT}})$ between adjacent table tennis balls. This approach allows us to evaluate the algorithm's ability to detect a table tennis ball $b_{i_x^{GT}}^{j_x^{GT}}$ when another ball $b_{i_y^{GT}}^{j_y^{GT}}$ has already been annotated.

Given that the algorithm uses ResNet as its backbone network, resulting in relatively coarse image granularity, we define a relationship $R(b_{i_x^{GT}}^{j_x^{GT}}, b_{i_y^{GT}}^{j_y^{GT}})$ as True Positive if the bounding box of a detected table tennis ball $b_{i_y}^{j_y}$ with a confidence score above a threshold has an IOU(Intersection over Union) greater than 0.3 with the annotated bounding box of $b_{i_y^{GT}}^{j_y^{GT}}$. Conversely, the relationship is labeled as False Positive if the IOU is below this threshold. If no table tennis ball $b_{i_y}^{j_y}$ is detected, the relationship is marked as False Negative. It is worth noting that we did not retrain the SOT model and instead used the published open-source model directly.

Comparison with Direct Use of Graph Attention Networks (GAT): As an alternative to our GMP message-passing network, we selected a three-layer GAT model for comparison and trained and evaluated it using the same approach as described in Section 4.2.1. It is noteworthy that, due to the slightly faster computation speed of the GAT model compared to our GMP model, we allocated more feature dimensions to the GAT model. Specifically, the input node feature dimensions for each layer of the three-layer GMP model were 70, 96, and 128, respectively. In contrast, the input node feature dimensions for the three-layer GAT model were 438, 502, and 566, respectively.

Furthermore, we compared the performance of the T-FORT algorithm with the FMO algorithm on a subset of videos from the dataset. We compiled statistics on the metrics of these videos and included them in the tables for reference. All the results are presented in Table 4.

Analysis

Experimental Factors and Error Sample Analysis: Reviewing the hypothesis proposed in Section 3.2, our discriminator is effectively regressing $\log(p(T_u))$, making decisions on potential table tennis balls in videos based on both image and motion features. In practical experiments, we observed significant differences in the trajectories and visual morphologies of table tennis balls captured from different camera angles, which are clearly reflected in our experimental data. Taking GMP(0.4–0.6) as an example, we analyzed the average results from three camera angles: Side-Facing, Player-Facing, and Directly-Facing, as summarized in Table 5.

In the Player-Facing view, the table tennis balls appear large in size, with distinct image features and significant contrast against the background due to the proximity to the table tennis table, making them easier to track. In the Side-Facing view, the movement of the table tennis balls exhibits strong regularity (primarily lateral movement). Although the balls may appear small and their image features blurry in some videos, the algorithm still achieves good tracking performance. However, in the Directly-Facing view, the trajectory of the table tennis balls is irregular, and due to interference and occlusion from the players, the balls have low contrast with the background noise, making them the most challenging to track. Consequently, the algorithm performs the worst

Algorithm Name	Train&Test Set						Validation Set					
	Target			Relationship			Target			Relationship		
	Pre.	Rec.	F1.	Pre.	Rec.	F1.	Pre.	Rec.	F1.	Pre.	Rec.	F1.
YOLOv8(conf = 0.01)[1]	0.853	0.744	0.795	N/A	N/A	N/A	0.551	0.738	0.631	N/A	N/A	N/A
YOLOv8(conf = 0.50)[1]	0.924	0.647	0.761	N/A	N/A	N/A	0.838	0.576	0.683	N/A	N/A	N/A
YOLOv8(0.01 − 0.50)[2]	0.908	0.685	0.781	N/A	N/A	N/A	0.773	0.641	0.701	N/A	N/A	N/A
SOT(0.05 − 0.95)[3]	N/A	N/A	N/A	0.832	0.705	0.763	N/A	N/A	N/A	0.792	0.483	0.6
GMP(0.40 − 0.60)[4]	0.88	0.946	0.912	0.865	0.927	0.895	0.981	0.946	0.963	0.912	0.938	0.925
GAT(0.40 − 0.60)[4]	0.849	0.907	0.877	0.837	0.883	0.859	0.875	0.863	0.869	0.834	0.813	0.823
T − FORT[5]	0.782	0.856	0.818	0.767	0.838	0.801	N/A	N/A	N/A	N/A	N/A	N/A
FMO[5]	0.711	0.592	0.646	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A

Table 4. Experimental Results of the Algorithm on our datasets. Pre., Rec., and F1. are abbreviations for Precision, Recall, and F1-Score, respectively. [1] conf=0.01 Means we set the confidence threshold of detection in YOLOv8 as 0.01. [2] The average result of YOLOv8 when threshold are set as 0.01,0.05,0.10,...,0.45,0.50. We select these thersholds because YOLOv8's F1-Score decrease sharply when threshold < 0.5 while keep consistent between 0.01 and 0.50. [3] The average result of SOT when threshold are set as 0.05,0.10,...,0.90,0.95. [4] The average result of GAT or GMP when threshold are set as 0.40,0.45,...,0.60. [5] When considering these algorithms, we only included the videos with algorithm results in their paper.

Algorithm Name	Target			Relationship		
	Precision	Recall	F1-Score	Precision	Recall	F1-Score
Side-Facing	0.862	0.954	0.906	0.853	0.941	0.895
Player-Facing	0.950	0.953	0.952	0.930	0.915	0.922
Directly-Facing	0.862	0.866	0.864	0.827	0.837	0.832
Total	0.880	0.946	0.912	0.865	0.927	0.895

Table 5. Tracking Performance under Different Views.

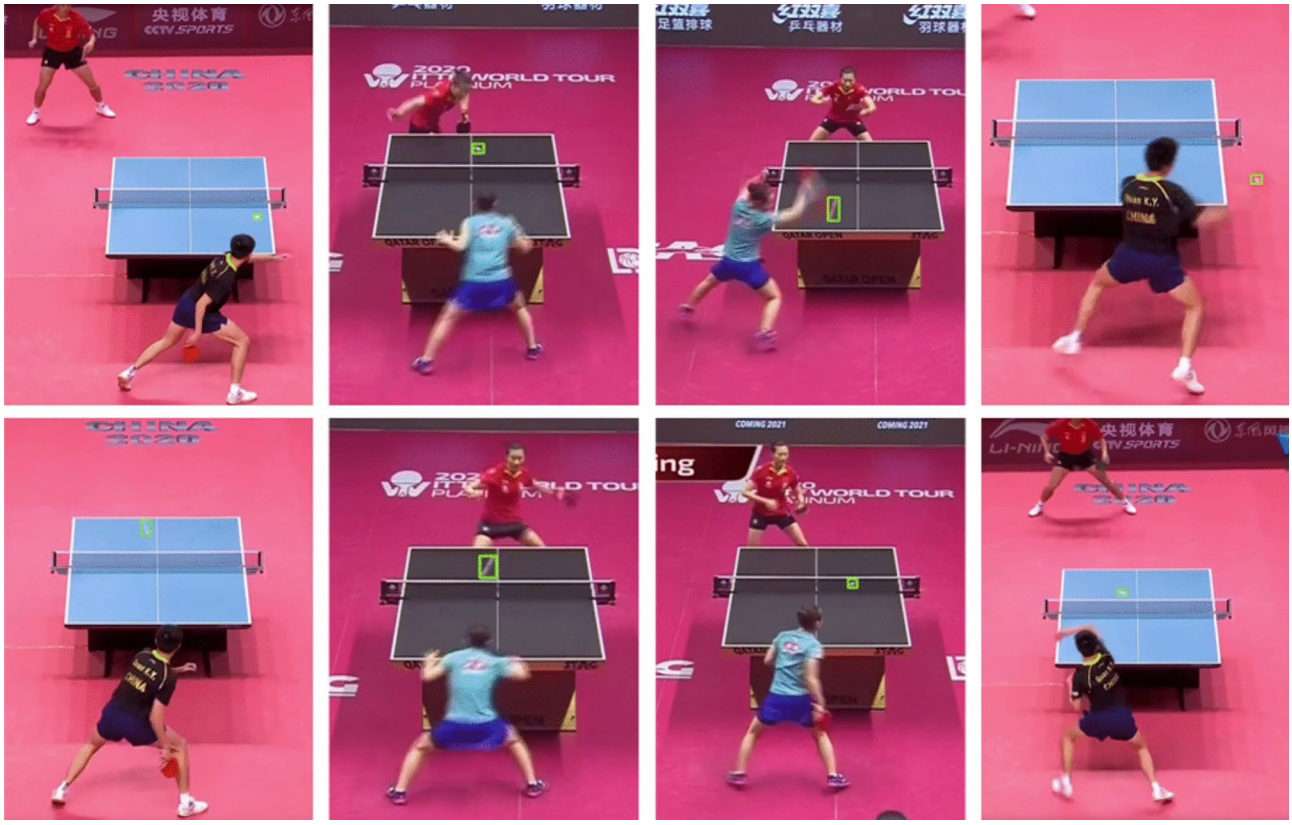


Fig. 19. Successfully detected table tennis balls in Directly-Facing view (marked with green boxes).

in this view. The table tennis ball samples successfully detected from the three perspectives of Directly-Facing, Player-Facing, and Side-Facing are shown in Figure 19, 20, and 21, respectively.

Analysis of False Negative Targets: The algorithm simply utilizes the HOG image operator, focusing solely on local image features of candidate targets without considering more peripheral or even global image features. The lack of semantic image features results in some table tennis balls that are not distinct from the background or are interfered with by the players' movements lacking the necessary features for detection, leading to them being missed by the algorithm. Additionally, due to the limitations of the differential detection algorithm, some table tennis balls fail to be accurately outlined with candidate boxes, resulting in the calculation of image features deviating from normal values and causing the subsequent algorithm to overlook these balls. Examples of table tennis balls falsely labeled as negative is shown Figure 22 .

Analysis of False Positive Targets: These targets primarily arise from background noise, local image features and motion trajectories generated by athletes' movements that closely resemble table tennis balls. Relying solely on local image and motion features of the table tennis balls makes it difficult to discern them accurately. Utilizing sufficiently discriminated data and image features with deep semantic information can assist in resolving some of these issues. Examples of table tennis balls falsely labeled as positive is shown Figure 23. Notably, the occurrence of these False Positive targets and trajectories is typically brief. Therefore, directly filtering out trajectories T_u with shorter lengths can significantly reduce the number of such targets. Similar strategies have also been proposed in[6]and[7]. Results of Filtering Out Short Trajectories is shown in Table A3 of Appendix A.



Fig. 20. Successfully detected table tennis balls in Player-Facing view (marked with green boxes).



Fig. 21. Successfully detected table tennis balls in Side-Facing view (marked with green boxes).

Model Parameters and Inference Speed: Our proposed method not only exhibits excellent performance but also boasts a significantly small number of parameters, totaling merely 0.8M. In contrast, the YOLOv8n model possesses a substantial parameter count of 3.2M. This is partly due to our simplified approach of utilizing the HOG operator as the image feature extractor, without designing a specialized feature extraction network for small targets. Another contributing factor is the nature of graph neural networks, where model parameters are reused across nodes and edges during inference, potentially leading to the neglect of challenging and infrequently occurring samples.

Regarding inference speed, we conducted tests on Side-Facing Professional athlete Competition videos in both 1080P and 720P resolutions. The testing environment consisted of a docker container equipped with a Teala T4 graphics card (Memory 16GB), CPU limitations ranging from 4–16 cores, and memory restrictions between 8192-32768MB. The running time for each component of our algorithm is summarized in Table 6.

Overall, inference speed is not a strength of our approach. Under the same testing conditions, YOLOv8's preprocess, inference, and postprocess stages averaged 1.8ms, 16.2ms, and 2.1ms, respectively, resulting in an



Fig. 22. Some examples of table tennis balls falsely labeled as negative (marked with blue boxes).

average frame time of 20.1ms and an inference performance exceeding 45fps (49.7fps). Furthermore, the running time of our solution is sensitive to the resolution of the video images, as the time spent by the detector is directly proportional to the image resolution. Nevertheless, the current performance of our algorithm is sufficient to meet the needs of offline inference services for users. If we consider only the Graph Inference component, our model can achieve a performance of approximately 10-20fps (without utilizing half-precision acceleration for inference in our tests). With optimizations to the detector, such as employing separate processes or adopting alternative faster solutions, our approach has the potential to achieve real-time tracking and detection.

Feasibility of Widespread Application: We downloaded a brief tennis match video (consisting of 155 frames) from the Wimbledon Tennis Championships official website[49]. We brutally applied the model previously used for table tennis tracking and the same environment utilized for table tennis tactical analysis to detect tennis balls in the video, subsequently analyzing them in a similar manner. The analysis results presented in Table 7 demonstrate that even with a model, method, and technical solution specifically designed for table tennis, a certain level of effectiveness can be achieved in a tennis scenario (Examples of successfully detected tennis ball is shown in Figure 24). Based on this, we can boldly hypothesize that our proposed technical solution, with

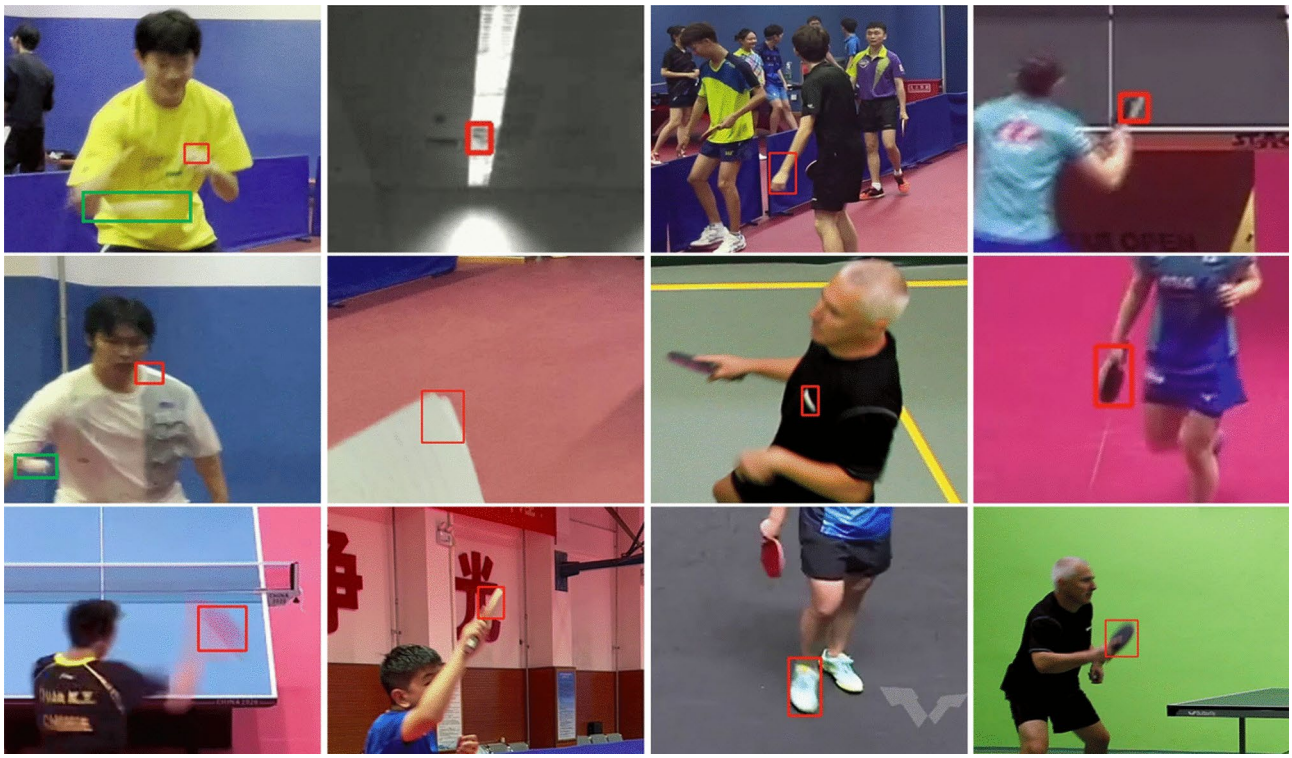


Fig. 23. Some examples of table tennis balls falsely labeled as positive (marked with red boxes).

	1080P		720P	
Stage of the Algorithm	Time Cost (s/frame)	fps	Time Cost (s/frame)	fps
Detector	0.34	2.94	0.08	11.81
Preprocess for Graph	0.08	13.18	0.02	52.33
Graph Inference	0.12	8.07	0.05	19.01
Organize Result	0.03	35.34	0.01	88.50
Total	0.57	1.76	0.17	5.96

Table 6. Tracking Performance under Different Views.

	Precision	Recall	F1-Score
Tennis Target	0.882	0.606	0.719
Tennis Relationship	0.786	0.473	0.591

Table 7. Analysis Results for Tennis Match Scenarios. The definitions of "Target" and "Relationship" in the table are identical to those in table tennis. Please refer to Section 4.2.1 for details.

targeted modifications and training, can be applied to other ball sports such as tennis, badminton, football, basketball, volleyball, and so on. Additionally, the GMP model itself can be utilized for similar problems such as link discovery in graphs.

Conclusion and future works

In this paper, we addressed the challenging problem of tracking fast-moving table tennis balls, which are small-sized, rapidly moving, and have non-significant image features. To tackle this, we employed a combined approach of detection and discrimination. Specifically, for the discriminator, we proposed a novel graph neural network called GMP, which features a unique message passing mechanism tailored to address the target tracking problem. Our experimental results demonstrated that our algorithm performs well in various scenarios within the dataset and holds promise for application to other high-speed target tracking problems.

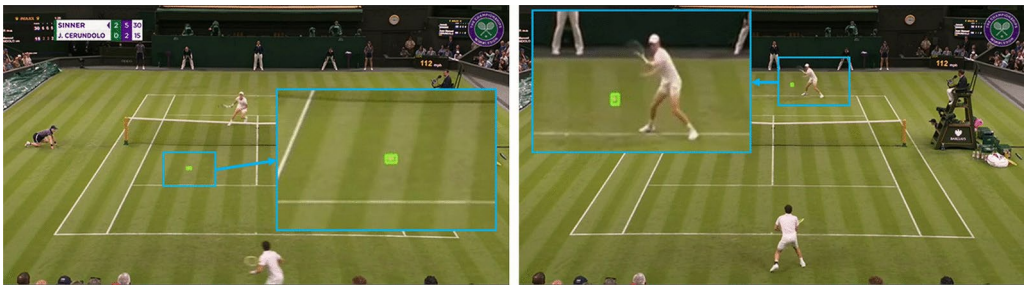


Fig. 24. The tennis ball successfully detected in a tennis match scenario.

For the future works,we may improve some details in our method considering some problems and limitations. Firstly, consider the aspect of image features. We solely utilized simple HOG features. It is evident that the use of image features containing semantic information could further enhance performance. In terms of target detection, simple difference detection and connected component analysis may not work effectively in all scenarios and are susceptible to interference from athletes and video noise within the scene. Therefore, more accurate scene modeling and target detection algorithms could help improve the algorithm's effectiveness. Further more, it would be a promising research topic to track high-speed targets such as table tennis balls using the state-of-the-art Segment Everything[50] as a detector, while incorporating contextual information. Additionally, modifying and enhancing the performance of the algorithm proposed in this paper or directly applying it to other ball sports or tracking problems are viable research directions or tasks for future exploration.

Data availability

Data sets generated during the current study are available from the corresponding author on reasonable request. The natural gas production data are available from Drilling Info but restrictions apply to the availability of these data, which were used under license for the current study, and so are not publicly available.

Materials availability

Materials generated during the current study are available from the corresponding author on reasonable request.

Code availability

If needed, the code can be made openly accessible.

Received: 25 July 2024; Accepted: 14 November 2024
Published online: 26 November 2024

References

1. Qiu, X., Zhang, H., Wei, J., Liu, J.: Machine learning based movement analysis and correction for table tennis. In: 2022 IEEE 8th International Conference on Cloud Computing and Intelligent Systems (CCIS), pp. 150–154 (2022). IEEE
2. Zheng, Y., Zhou, W., Zou, T., Zhang, H.: A method for table tennis bat trajectories reconstruction with the fusion of human keypoint information. In: 2023 8th IEEE International Conference on Network Intelligence and Digital Content (IC-NIDC), pp. 71–75 (2023). IEEE
3. Yu, B., Wei, J., Zhang, H., Jin, Z., Yu, Y., Wang, Q., Liu, J.: A machine learning based method to establish normative action model for table tennis. In: 2023 8th IEEE International Conference on Network Intelligence and Digital Content (IC-NIDC), pp. 91–95 (2023). IEEE
4. Cores, D., Brea, V. M. & Mucientes, M. Spatiotemporal tubelet feature aggregation and object linking for small object detection in videos. *Applied Intelligence* **53**(1), 1205–1217 (2023).
5. Rozumnyi, D., Kotera, J., Sroubek, F., Novotny, L., Matas, J.: The world of fast moving objects. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, pp. 5203–5211 (2017)
6. Zou, T., Wei, J., Zhang, H., Jin, Z., Yu, Y., Liu, J.: T-fort: A tree-based fast-moving object trajectory tracking algorithm for table tennis. In: 2023 8th IEEE International Conference on Network Intelligence and Digital Content (IC-NIDC), pp. 76–80 (2023). IEEE
7. Jin, Z., Zou, T., Sun, D., Yang, Y. & Liu, J. A semantic tree-based fast-moving object trajectory tracking algorithm for table tennis. *International Journal on Semantic Web and Information Systems (IJSWIS)* **20**(1), 1–17 (2024).
8. Hu, Y., Niu, A., Sun, J., Zhu, Y., Yan, Q., Dong, W., Woźniak, M., Zhang, Y.: Dynamic center point learning for multiple object tracking under severe occlusions. Knowledge-Based Systems, 112130 (2024)
9. Voeikov, R., Falaleev, N., Baikulov, R.: Ttnet: Real-time temporal and spatial video analysis of table tennis. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops, pp. 884–885 (2020)
10. Redmon, J., Divvala, S., Girshick, R., Farhadi, A.: You only look once: Unified, real-time object detection. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, pp. 779–788 (2016)
11. Terven, J., Córdova-Esparza, D.-M. & Romero-González, J.-A. A comprehensive review of yolo architectures in computer vision: From yolov1 to yolov8 and yolo-nas. *Machine Learning and Knowledge Extraction* **5**(4), 1680–1716 (2023).
12. Jocher, G., Chaurasia, A., Qiu, J.: Ultralytics YOLO (2023). https://github.com/ultralytics/ultralytics
13. Li, B., Wu, W., Wang, Q., Zhang, F., Xing, J., Yan, J.: Siamrpn++: Evolution of siamese visual tracking with very deep networks. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp. 4282–4291 (2019)
14. Velickovic, P. et al. *Graph attention networks. stat* **1050**(20), 10–48550 (2017).

15. Groos, D., Ramampiaro, H. & Ihlen, E. A. Efficientpose: Scalable single-person pose estimation. *Applied intelligence* **51**(4), 2518–2533 (2021).
16. Yu, H., Chen, D., Tao, Y., Yu, B., He, G.: Analysis and guidance for standing broad jump based on artificial intelligence. In: 2023 8th IEEE International Conference on Network Intelligence and Digital Content (IC-NIDC), pp. 81–85 (2023). IEEE
17. Li, W. Analyzing the rotation trajectory in table tennis using deep learning. *Soft Computing* **27**(17), 12769–12785 (2023).
18. Zhu, Q. & Deng, H. Spatial adaptive graph convolutional network for skeleton-based action recognition. *Applied Intelligence* **53**(14), 17796–17808 (2023).
19. Wei, J., Yu, B., Zhang, H., Liu, J.: Skeleton based graph convolutional network method for action recognition in sports: A review. In: 2023 8th IEEE International Conference on Network Intelligence and Digital Content (IC-NIDC), pp. 66–70 (2023). IEEE
20. Hu, M., Qiu, X., Zou, T., Zhang, H., Jin, Z., Yu, Y.: Research and implementation of table tennis sport simulator based on physical theory. In: 2023 8th IEEE International Conference on Network Intelligence and Digital Content (IC-NIDC), pp. 86–90 (2023). IEEE
21. Rong, Z. Optimization of table tennis target detection algorithm guided by multi-scale feature fusion of deep learning. *Scientific Reports* **14**(1), 1401 (2024).
22. Zita, A., Šroubek, F.: Tracking fast moving objects by segmentation network. In: 2020 25th International Conference on Pattern Recognition (ICPR), pp. 10312–10319 (2021). IEEE
23. Ning, T., Wang, C., Fu, M. & Duan, X. A study on table tennis landing point detection algorithm based on spatial domain information. *Scientific Reports* **13**(1), 20656 (2023).
24. Zhang, X., Zhang, T., Yang, Y., Wang, Z., Wang, G.: Real-time golf ball detection and tracking based on convolutional neural networks, 2808–2813 (2020). IEEE
25. Pal, S. K., Pramanik, A., Maiti, J. & Mitra, P. Deep learning in multi-object detection and tracking: state of the art. *Applied Intelligence* **51**, 6400–6429 (2021).
26. Izquierdo, R. et al. Vehicle trajectory prediction on highways using bird eye view representations and deep learning. *Applied Intelligence* **53**(7), 8370–8388 (2023).
27. Rodriguez-Lozano, F. J., Gámez-Granados, J. C., Martínez, H., Palomares, J. M. & Olivares, J. 3d reconstruction system and multiobject local tracking algorithm designed for billiards. *Applied Intelligence* **53**(19), 21543–21575 (2023).
28. Zhang, Y., Wang, T., Liu, K., Zhang, B. & Chen, L. Recent advances of single-object tracking methods: A brief survey. *Neurocomputing* **455**, 1–11 (2021).
29. Fan, H., Bai, H., Lin, L., Yang, F., Chu, P., Deng, G., Yu, S., Harshit, Huang, M., Liu, J., : Lasot: A high-quality large-scale single object tracking benchmark. International Journal of Computer Vision 129, 439–461 (2021)
30. Scarselli, F., Gori, M., Tsoi, A. C., Hagenbuchner, M. & Monfardini, G. The graph neural network model. *IEEE transactions on neural networks* **20**(1), 61–80 (2008).
31. Dwivedi, V.P., Bresson, X.: A generalization of transformer networks to graphs. arXiv preprint arXiv:2012.09699 (2020)
32. Ullah, I., Manzo, M., Shah, M. & Madden, M. G. Graph convolutional networks: analysis, improvements and results. *Applied Intelligence* **52**(8), 9033–9044 (2022).
33. Sola, F., Ayala, D., Hernández, I. & Ruiz, D. Deep embeddings and graph neural networks: using context to improve domain-independent predictions. *Applied Intelligence* **53**(19), 22415–22428 (2023).
34. Raabe, D., Nabben, R. & Memmert, D. Graph representations for the analysis of multi-agent spatiotemporal sports data. *Applied Intelligence* **53**(4), 3783–3803 (2023).
35. Yu, S., Zeng, Y., Pan, Y. & Chen, B. Discovering a cohesive football team through playersâ€™ attributed collaboration networks. *Applied Intelligence* **53**(11), 13506–13526 (2023).
36. Nouraie, M., Eslahchi, C. & Baca, A. Intelligent team formation and player selection: a data-driven approach for football coaches. *Applied Intelligence* **53**(24), 30250–30265 (2023).
37. Jahangard, S., Hayat, M., Rezatofighi, H.: Real-time trajectory-based social group detection. In: 2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), pp. 1901–1908 (2023). IEEE
38. Cetintas, O., Brasó, G., Leal-Taixé, L.: Unifying short and long-term tracking with graph hierarchies. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp. 22877–22887 (2023)
39. Bashar, M., Islam, S., Hussain, K.K., Hasan, M.B., Rahman, A., Kabir, M.H.: Multiple object tracking in recent times: A literature review. arXiv preprint arXiv:2209.04796 (2022)
40. Wei, L., Xi, Z., Hu, Z. & Sun, H. Graph attention information fusion for siamese adaptive attention tracking. *Applied Intelligence* **53**(2), 2068–2087 (2023).
41. Singh, D. & Srivastava, R. Graph neural network with rnns based trajectory prediction of dynamic agents for autonomous vehicle. *Applied Intelligence* **52**(11), 12801–12816 (2022).
42. Liu, R., Calafiura, P., Farrell, S., Ju, X., Murnane, D.T., Pham, T.M.: Hierarchical graph neural networks for particle track reconstruction. In: 21th International Workshop on Advanced Computing and Analysis Techniques in Physics Research: AI Meets Reality, vol. 3 (2023)
43. Brasó, G., Cetintas, O. & Leal-Taixé, L. Multi-object tracking and segmentation via neural message passing. *International Journal of Computer Vision* **130**(12), 3035–3053 (2022).
44. Dai, P., Weng, R., Choi, W., Zhang, C., He, Z., Ding, W.: Learning a proposal classifier for multiple object tracking. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp. 2443–2452 (2021)
45. He, J., Huang, Z., Wang, N., Zhang, Z.: Learnable graph matching: Incorporating graph partitioning with deep feature learning for multiple object tracking. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp. 5299–5309 (2021)
46. Dong, W. et al. Improving performance and efficiency of graph neural networks by injective aggregation. *Knowledge-Based Systems* **254**, 109616 (2022).
47. Ke, Q. et al. Apgvae: Adaptive disentangled representation learning with the graph-based structure information. *Information Sciences* **657**, 119903 (2024).
48. Liu, Y., Ao, X., Qin, Z., Chi, J., Feng, J., Yang, H., He, Q.: Pick and choose: a gnn-based imbalanced learning approach for fraud detection. In: Proceedings of the Web Conference 2021, pp. 3168–3177 (2021)
49. Jannik Sinner-Best Point of Wimbledon 2023. Website Videoshare https://www.wimbledon.com/en GB/video/index.html (2024)
50. Kirillov, A., Mintun, E., Ravi, N., Mao, H., Rolland, C., Gustafson, L., Xiao, T., Whitehead, S., Berg, A.C., Lo, W.-Y.,: Segment anything. In: Proceedings of the IEEE/CVF International Conference on Computer Vision, pp. 4015–4026 (2023)

Author contributions

Conceptualization. Tianjian Zou and Jun Liu. Methodology, Software, Visualization and Writing - Original Draft. Tianjian Zou. Validation, Formal analysis, Investigation, Data Curation. Jiangning Wei, Bo Yu, Xinzhu-Qiu, LiuJun and Tianjian Zou. Resources. Hao Zhang and Xu Du. Writing - Review & Editing, Supervision and Project administration. Jun Liu.

Declarations

Competing interests
The authors declare no competing interests.

Consent for publication
We confirm that we have obtained informed consent from all subjects and/or their legal guardians for the publication of personal information/images in openly accessible online publications, and that data collection was only carried out after obtaining their informed consent.

Informed consent
We confirm that all methods were carried out in accordance with relevant guidelines and regulations. We confirm that all experimental protocols were approved by "Working Committee on Ethics and Governance of Artificial Intelligence" of "Chinese Association for Artificial Intelligence". We confirm that informed consent was obtained from all subjects and/or their legal guardian(s).

Additional information
Supplementary Information The online version contains supplementary material available at https://doi.org/10.1038/s41598-024-80056-3.

Correspondence and requests for materials should be addressed to J.L.

Reprints and permissions information is available at www.nature.com/reprints.

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by-nc-nd/4.0/.

© The Author(s) 2024